



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Diseño e implementación de un sistema RAG (Retrieval-Augmented Generation) en un LLM para la construcción de reglas YARA en la detección de malware

Trabajo de investigación/grado para optar el título de
Magister en Ciencias de la Información y las Comunicaciones

César Iván Quintero Garzón

Asesor

Ing. Edwar Jacinto Gómez MSC

Bogotá D. C. – Colombia

Mayo de 2026

Contenido

1.	INTRODUCCIÓN	4
2.	DESARROLLO DEL PROYECTO	8
2.1	CAPÍTULO 1: REVISIÓN DE BASES DE DATOS DE MALWARE Y SELECCIÓN DE MALPEDIA	8
2.1.1	Clasificación de bases de datos de malware	8
2.1.2	Limitaciones identificadas en las bases de datos analizadas.....	10
2.1.3	Justificación de la selección de Malpedia	10
2.1.4	Acceso a Malpedia	11
2.2	CAPÍTULO 2: REVISIÓN DE BASES DE DATOS DE MALWARE Y SELECCIÓN DE MALPEDIA	12
2.2.1	Delimitación del proyecto: enfoque en ransomware.....	12
2.2.2	Caracterización por medio de análisis estático	16
2.3	CAPÍTULO 3: Diseño e implementación del sistema RAG para la generación de reglas YARA 18	
2.3.1	Fundamentos del RAG	19
2.3.2	Componentes del RAG propuesto	20
2.3.3	Componentes tecnológicos de la implementación	21
2.3.4	Ciclo de funcionamiento del RAG	24
2.3.5	DeepSeek como LLM para el proyecto y selección del modelo DeepSeek 8B cuantizado	27
2.3.6	Infraestructura computacional utilizada.....	29
2.4	CAPÍTULO 4: Diseño del prompt para la generación de reglas YARA en el sistema RAG 32	
2.4.1	Prompt inicial.....	32
2.4.2	Evaluación mediante self-reflection	34
2.5	CAPÍTULO 5: Métricas para la evaluación de las reglas YARA RAG y LLM sin RAG 35	
2.5.1	Enfoques para evaluar	36

2.6	CAPÍTULO 6: Pruebas y evaluación de las reglas YARA RAG y LLM sin RAG	39
3.	CONCLUSIONES.....	46
4.	PRINCIPALES APORTES.....	49
4.1	Aporte metodológico	49
4.2	Aporte técnico.....	49
4.3	Aporte comparativo.....	50
4.4	Aporte en escalabilidad.....	50
4.5	Aporte en soluciones Security Operations Center (SOC).....	50
5.	RECOMENDACIONES Y FUTUROS TRABAJOS	52
6.	REFERENCIAS.....	55

1. INTRODUCCIÓN

En la actualidad, la seguridad informática enfrenta desafíos críticos debido al incremento sostenido del malware y a la evolución constante de los adversarios, quienes desarrollan técnicas para evadir los mecanismos de detección (Mohammadinodooshan et al., 2019). Por consiguiente, la identificación efectiva de estas amenazas es imperativa para salvaguardar la integridad de los sistemas informáticos en general, junto con las infraestructuras de red y los activos de datos sensibles (Patel et al., 2023).

Es importante entender el funcionamiento del malware y sus componentes lo cual permite su ejecución, propagación y persistencia dentro de un sistema comprometido. Este está conformado por un conjunto de características internas que permiten identificar una muestra maliciosa. Entre estas podemos encontrar las cadenas de texto (strings), librerías importadas dlls, llamadas a APIs, secciones del ejecutable, métodos de ofuscación, dominios, direcciones IP, hashes y comunicación con servidores remotos (Das et al., 2023).

Para el caso de un ransomware, la anatomía del malware suele incluir funciones relacionadas con el cifrado de archivos, eliminación de copias de seguridad, persistencia en el sistema afectado, enumeración de directorios y comunicación con infraestructura de comando y control C&C. También, muchas familias incorporan mecanismos que dificultan el análisis, así como técnicas de evasión que hacen más difícil su detección. El estudio de estas componentes resulta importante para identificar patrones característicos y construir mecanismos de detección efectivos (Alchi & Dodiya, 2023).

Estos componentes mencionados como características relevantes de un malware corresponden a los Indicadores de Compromiso (IoC) los cuales son evidencia técnica que demuestran la actividad maliciosa de un archivo dentro de un sistema. Estos IoC en ciberseguridad permiten detectar y responder ante incidentes de malware y ataques en la infraestructura informática (Ocãnoia et al., 2025).

Entre los IoCs más comunes se encuentran:

- Hashes de archivos (MD5, SHA1, SHA256)
- Direcciones IP maliciosas

- Dominios asociados a infraestructura de ataque
- URLs utilizadas para descarga o comunicación
- Nombres de archivos sospechosos
- Claves de registro
- Procesos anómalos
- Strings específicas presentes en malware

También hay IoC que permiten identificar características asociadas a campañas de ransomware vinculadas a variantes específicas e infraestructura de comando y control. Todos estos indicadores permiten construir reglas YARA debido a que proporcionan patrones específicos de los archivos maliciosos que pueden permitir su detección.

Las reglas YARA se consolidan como una de las herramientas más efectivas para dicha labor, al permitir la identificación de patrones específicos en archivos y procesos en ejecución. No obstante, la autoría manual de estas reglas presenta limitaciones significativas: el proceso es laborioso y consume tiempo, requiere un nivel alto de experiencia especializada y restringe la escalabilidad del análisis, lo cual impacta directamente en la capacidad de detección de nuevas amenazas (Naik et al., 2019).

Las reglas YARA constituyen un componente esencial en la infraestructura de ciberseguridad, herramientas diseñadas principalmente para la identificación de patrones distintivos tanto en archivos estáticos como en procesos en ejecución (Siddabathula et al., 2023). Su correcta configuración es vital para monitorear entornos y detectar actividades sospechosas. No obstante, esta dependencia introduce desafíos significativos (Xu & Qiao, 2022). Actualmente, el desarrollo y mantenimiento de reglas YARA—ya sea mediante creación manual o automatización—enfrenta limitaciones críticas: es difícil garantizar la precisión en la identificación, la capacidad de detección de cambios y la gestión eficiente de la vasta cantidad de información sobre programas maliciosos (Si et al., 2022). A esto se suma la naturaleza dinámica del malware, la escasez de muestras nuevas y la frecuente inexactitud en las alertas, que resulta en falsos positivos o falsos negativos. Estos factores comprometen la efectividad de los sistemas de detección, dejando vulnerabilidades explotables por los cibercriminales, lo que incrementa los riesgos de seguridad y dificulta la respuesta ante ataques por parte de los equipos de informática (Patil et al., 2025).

Este proyecto pretende abordar dichas limitaciones, dado que actualmente el diseño de reglas YARA, ya sea mediante intervención manual o automatización convencional, presenta desafíos críticos en precisión, adaptabilidad y gestión de información (Naik, Jenkins, Cooke, et al., 2020). La dinámica cambiante del malware, combinada con la escasez de muestras actualizadas y la presencia de errores de clasificación como falsos positivos o negativos, compromete la eficacia de los sistemas de detección, exponiendo vulnerabilidades explotables por cibercriminales y dificultando la respuesta de los equipos de seguridad. Ante este escenario, resulta imperativo el desarrollo de herramientas modernas que optimicen la creación automática de reglas. Por ello, la propuesta investiga y construye un sistema basado en la metodología Retrieval-Augmented Generation (RAG) (Lewis et al., 2021), integrado con un modelo de lenguaje de gran escala (LLM). Este enfoque capitaliza la capacidad generativa de los LLMs (Renze & Guven, 2024) y la recuperación de información específica almacenada en colecciones de muestras analizadas, permitiendo generar reglas más exactas, robustas y adaptadas a la evolución de las amenazas (Jelodar et al., 2025).

Tomando en cuenta este escenario, se requiere crear y adaptar herramientas tecnológicas para construir reglas YARA de forma automática (Raff et al., 2020). Esto hará que sean mejores, más exactas y se ajusten mejor al cambio constante de este tipo de amenazas. Por eso, este proyecto planea diseñar y construir un sistema que use la metodología Retrieval-Augmented Generation (RAG) y, junto con un modelo de lenguaje de gran escala (LLM), tenga la capacidad de generar contenido, a lo que se suma la habilidad de encontrar información específica en una colección de muestras de malware ya analizadas (Zhang et al., 2025).

En relación con lo expuesto anteriormente, el objetivo general de esta investigación es diseñar, implementar y evaluar un sistema RAG (Pal Singh Shobhit, 2024) que combine un componente de recuperación basado en una base de datos de malware y un modelo de lenguaje grande, con el fin de generar reglas YARA automatizadas y de alta calidad para la detección de malware.

Para alcanzar este objetivo, se establecen metas específicas: recopilar y clasificar ejemplos de diversas familias de malware, con especial atención al ransomware; analizar exhaustivamente estos ejemplos mediante técnicas de análisis estático para extraer información relevante (Raff et al., 2017); construir una base de datos que respalde el diseño y la organización del sistema RAG; determinar los componentes del software responsables de la búsqueda de información y la

generación de respuestas; desarrollar métodos que permitan al modelo utilizar la información obtenida y, posteriormente, redactar reglas YARA que sean efectivas para la detección de malware (S. Li et al., 2023). Finalmente, se definen métricas para evaluar las reglas creadas, verificando su eficacia, su cobertura de diferentes tipos de malware y su capacidad real de detección, lo que permitirá contar con herramientas comparativas para evaluar el desempeño del sistema con distintos modelos de lenguaje ajenos al proceso de búsqueda de información (RAG).

Cabe resaltar que esta investigación busca conectar el estudio de la teleinformática y la ciberseguridad, ya que cada vez se necesita más recurrir a técnicas de inteligencia artificial para analizar y detectar amenazas. Unir RAG y LLM se plantea como una manera de automatizar la creación de reglas YARA para identificar intrusiones, disminuir la dependencia de los analistas de seguridad informática y responder de manera rápida ante un incidente. Por otra parte, este proyecto pretende ayudar tanto a la investigación académica como a la aplicación práctica, ofreciendo una herramienta que los programas de defensa de ciberseguridad puedan utilizar contra los programas maliciosos.

Finalmente, el presente documento muestra en el primer capítulo se identifican los diferentes repositorios públicos y/o privados de malware, la calidad y la cantidad de las muestras, así como su facilidad de acceso, con el fin de obtener los datos requeridos para la investigación. En el segundo capítulo se realizará una caracterización de las muestras de ransomware, mediante un análisis estático que permita extraer los indicadores de compromiso IoC y otras características especiales, a partir de la cual se construirá el repositorio. Para el tercer capítulo, se diseñará e implementará el RAG que indexará el repositorio de caracterización de ransomware. En el cuarto capítulo se investigarán y se ejecutarán diversas técnicas de prompting para obtener el mejor resultado en la creación de reglas YARA. En el quinto capítulo se determinarán las métricas de calidad, cobertura y efectividad de las reglas de YARA generadas por el RAG. Y por último, en el sexto capítulo se realizará una comparación con reglas YARA generadas por un LLM teniendo en cuenta distintas métricas establecidas con anterioridad.

2. DESARROLLO DEL PROYECTO

Para empezar, es importante revisar los repositorios de muestras de malware disponibles más relevantes. Ya que, la calidad y la cantidad de estas muestras afectarán directamente al sistema RAG en el que se está trabajando, porque constituyen el conjunto de información que se utilizará para encontrar y crear reglas YARA. Tener acceso a muestras reales es un punto neurálgico en este trabajo, por lo que es muy importante escoger el repositorio adecuado.

2.1 CAPÍTULO 1: REVISIÓN DE BASES DE DATOS DE MALWARE Y SELECCIÓN DE MALPEDIA

Para este capítulo, es fundamental examinar exhaustivamente las fuentes de datos y los repositorios más relevantes. La integridad, diversidad y estructura de estas bases de datos influyen directamente en la eficacia del sistema RAG. Esto se debe a que constituyen la base de conocimiento utilizada para identificar y desarrollar reglas de YARA. Obtener muestras auténticas es neurálgico y puede restringir el alcance del estudio, por lo que la selección de las fuentes de datos debe ser exhaustiva (Bilstein & Plohmann, 2019).

2.1.1 Clasificación de bases de datos de malware

2.1.1.1 Repositorios públicos abiertos

En este estudio se emplearon diversos repositorios públicos para obtener archivos de malware, seleccionados por su facilidad de acceso y la abundancia de muestras disponibles

- Malware Bazaar (abuse.ch): ofrece archivos de malware recientes y con información adicional (los códigos únicos de los archivos, palabras clave, e identifica el tipo de malware). Actualización de su base de conocimiento continua, pero con poco análisis.
- VirusShare: posee millones de archivos de malware en formato crudo. Dicho repositorio es adecuado para analizar múltiples archivos simultáneamente, pero carece de etiquetas

bien organizadas, además de revisión y soporte, lo que dificulta su uso directo en sistemas generativos (RAG).

- VirusTotal: es una plataforma reconocida que te permite buscar archivos de malware y ver los informes de múltiples programas antivirus. Aunque cuenta con una amplia base de datos de malware, requiere pago para acceder a todas sus funciones, lo que limita su uso en investigaciones universitarias.
- TheZoo: es un lugar para reunir malware para analizarlo en el ámbito educativo. Tiene problemas con las actualizaciones y con el soporte continuo.
- Malpedia: es una página web diseñada para estudiar malware y recopilar información. Su función principal es recopilar detalles técnicos de sus distintos tipos. Actúa como un almacén de amenazas informáticas, lo que permite a los expertos en seguridad identificar, analizar y compartir las características, los identificadores y el funcionamiento de los virus.

2.1.1.2 Bases de datos privadas o comerciales

Las plataformas de inteligencia sobre amenazas (TIPs) como FireEye, CrowdStrike y Kaspersky Threat Intelligence ofrecen información muy bien organizada y llena de detalles, aunque no siempre es fácil conseguirla. También existen las bases de datos internas, pertenecientes a las compañías que dan soporte de en el área de la seguridad informática. Es cierto que estas fuentes tienen datos de mucha calidad, pero para la investigación universitaria su utilización es limitada debido al costo, a los permisos que se necesitan y a ciertas leyes que lo impiden.

2.1.1.3 Datasets académicos

En el mundo académico se han creado colecciones de datos para problemas específicos. El conjunto de datos EMBER, por ejemplo, se usa para identificar tipos de software malicioso con aprendizaje automático. También está el conjunto de datos de Microsoft para el desafío de clasificación de programas maliciosos y el conjunto de datos Drebin, que se centra en sistemas Android. Estos conjuntos de datos normalmente están organizados de forma clara, pero presentan dos problemáticas principales: no se actualizan mucho cuando aparecen amenazas

nuevas y no ofrecen suficiente información detallada sobre cómo son y qué hacen las diferentes familias de software malicioso, lo cual, hace que no sean adecuados para crear reglas YARA.

2.1.2 Limitaciones identificadas en las bases de datos analizadas

Al revisar las colecciones de malware, se ven varias carencias que se repiten. Muchas bases de datos tienen ejemplos de programas maliciosos, pero no explican con detalle cómo actúa el malware. Con repositorios enormes como VirusShare, es necesario limpiarlos y ordenarlos, y para usar plataformas como VirusTotal para investigar a fondo, hay límites. Los conjuntos de datos que se utilizan en la investigación académica se quedan atrás porque el malware cambia sin parar. Además, no se pone mucho énfasis en tipos específicos de malware; y esto es importante para nuestro estudio porque nos centramos en el ransomware. Todas estas dificultades hacen que sea difícil crear una colección de datos útil para un sistema RAG, porque este sistema necesita no solo muchos datos, también que sean buenos, que estén bien organizados y que tengan información sobre su origen y significado.

2.1.3 Justificación de la selección de Malpedia

Después de comparar varias fuentes, se decide usar Malpedia¹ como la base de datos principal para este estudio. Malpedia es un lugar especializado donde se ordenan y clasifican las familias de malware, ofreciendo datos revisados y confirmados por especialistas. A diferencia de otras colecciones muy grandes de archivos, Malpedia organiza las muestras por familia, lo cual, lo hace ideal para crear reglas YARA, porque estas reglas similitudes para determinar qué tipo de malware y su de versión. Esto hace que esta fuente sea ideal para este proyecto, ya que permite describir el malware por familias. Además, Malpedia es bastante accesible para trabajos académica en comparación con plataformas de pago más cerradas.

De hecho, elegir Malpedia es totalmente acorde con la estructura del sistema mostrado anteriormente, porque se requiere una base de datos con tres características: encontrar información importante rápido, darle al modelo de lenguaje (LLM) información organizada, y hacer más fácil la creación de reglas YARA basadas en pruebas verdaderas.

¹ <https://malpedia.caad.fkie.fraunhofer.de/>

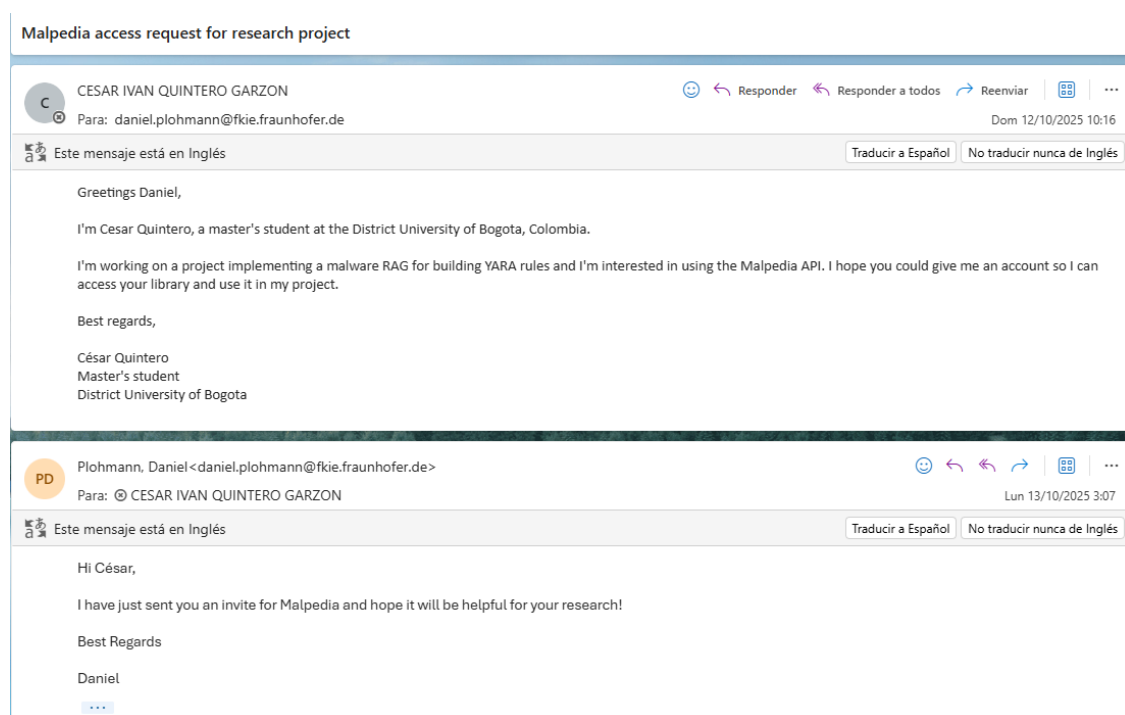
Así, Malpedia no es solamente un sitio para guardar archivos de malware, es también una fuente de información más amplia, de contexto, y eso es fundamental para sistemas tipo RAG, como el que requiere para el desarrollo de este proyecto.

2.1.4 Acceso a Malpedia

El acceso al repositorio de malware de Malpedia no es accesible directamente desde su página web, por lo cual se decide contactar directamente por email con Daniel Plohmann el cual es el administrador y creador de este proyecto obteniendo un resultado positivo.

Figura 1.

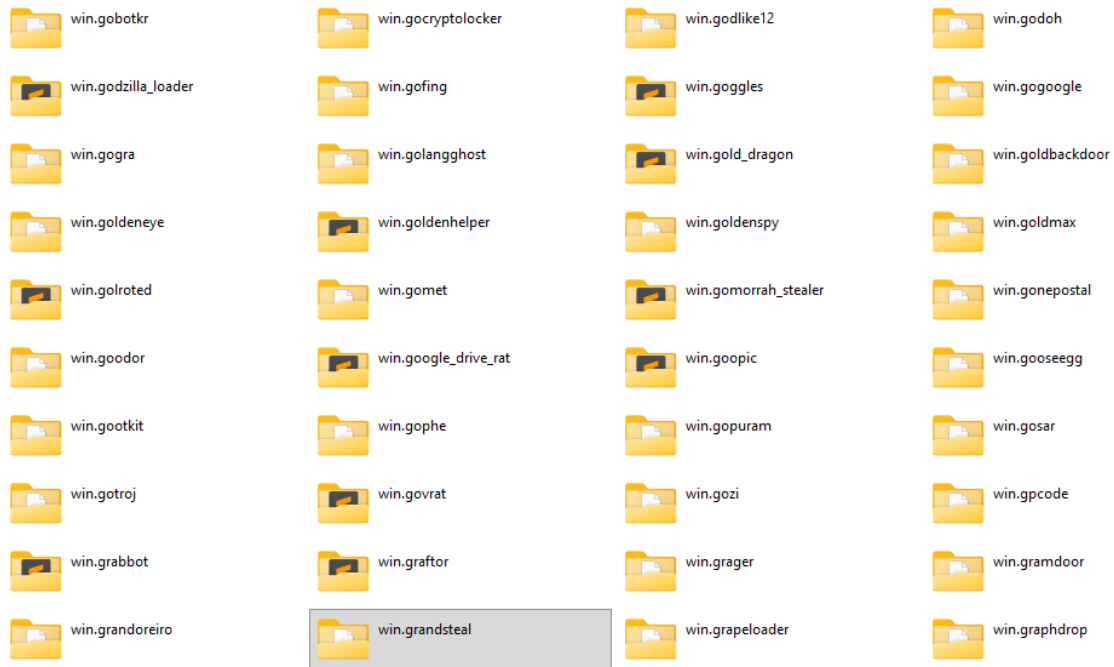
Solicitud de acceso al repositorio de Malpedia



El repositorio de malware descargado de Malpedia contiene diferentes muestras de archivos maliciosos organizados por la familia y fechas de obtención. Este repositorio pesa 22,4 GB, contiene 21.660 archivos y 11.869 carpetas.

Figura 2.

Muestra de carpetas del repositorio de Malpedia



2.2 CAPÍTULO 2: REVISIÓN DE BASES DE DATOS DE MALWARE Y SELECCIÓN DE MALPEDIA

2.2.1 Delimitación del proyecto: enfoque en ransomware

El proyecto se enfocará únicamente en el tipo de malware conocido como ransomware. Por las siguientes razones:

Primero, el ransomware es actualmente una de las mayores amenazas en seguridad informática, por el daño que ejerce a nivel económico, por lo rápido que puede propagarse y por cómo está cambiando continuamente. Además, la rapidez con que surgen nuevas formas y métodos que ocultan su carácter malicioso dificulta encontrarlos mediante las técnicas típicas de detección.

Segundo, al analizar un solo tipo de malware, se reduce la diversidad de los archivos a estudiar, se identifican más fácilmente los puntos en común entre ellos (Mr. Shreyas Biju Nair et al., 2024),

se obtienen mejores reglas YARA y se mejora la velocidad del sistema RAG al tener que ocuparse de un conjunto de datos más limitado.

Por último, reducir el alcance del análisis permitirá crear una colección de archivos más ordenada y coherente, enfocada en familias particulares de ransomware. Esto es importantísimo para encontrar información en una etapa posterior.

Se decide usar las muestras dentro del repositorio de Malpedia de las diferentes familias de ransomware que han sido identificadas a lo largo de los años, teniendo en cuenta desde las más famosas hasta variantes más específicas o menos conocidas; entre ellas se encuentran 8base, agenda, akira, archiveus, babuk, bad rabbit, bianlian, blackbasta, blackbyte, blackcat, blackmatter, blacksuit, cerber, clop, conti, cryptolocker, cryptowall, crysis, ctb-locker, darkside, dharma, doppelpaymer, egregor, esxiargs, gandcrab, gpcode, hermes, hive, jigsaw, killsec, krotten, lockbit, lockergoga, locky, makop, maze, medusa, mountlocker, nefilim, netwalker, notpetya, petya, phobos, play, pysa, qilin, ragnarlocker, ransomhub, reveton, revil, rhysida, royal, ryuk, samsam, snatch, sodinokibi, stop/djvu, svpeng, teslacrypt, thanos, torrentlocker, trojan.winlock, wannacry, wastedlocker, zeppelin y whiterabbit.

El total de muestras extraídas de las familias de ransomware del repositorio de Malpedia son 698 las cuáles serán las procesadas en el proyecto.

Tabla 1.

Muestras de familias de ransomware según sistema operativo encontradas en Malpedia

Windows	54
Linux	17

Tabla 2.

Cantidad de muestras por cada familia de ransomware extraídas de Malpedia

Nombre de ransomware	Número de archivos
win.nefilim	7
elf.royal_ransom	1
win.maze	49
elf.rhysida	1
win.locky_loader	2
win.makop	1
win.cerber	21
win.gandcrab	36
win.play	1
win.zeppelin	9
win.lockergoga	8
elf.blackcat	2
win.teslacrypt	19
win.locky	52
win.dharma	2
win.blacksuit	2
win.petya	11
win.clop	17
elf.blackbasta	2
win.cryptolocker	9
win.royal_ransom	1
win.ransomhub	1
win.eternal_petya	8
win.medusa_http	7
win.hive	9
elf.clop	1
elf.conti	1
elf.ragnarlocker	1
win.lockbit	8
win.medusa	1
win.reveton	2
win.torrentlocker	10
elf.darkside	3
elf.akira	2
win.gpcode	30
elf.blackmatter	1

win.rhysida	9
win.blackmatter	7
win.akira	2
win.conti	14
win.doppelpaymer	12
win.revil	53
win.samsam	7
win.blackcat	6
elf.revil	4
elf.babuk	6
win.jigsaw	2
win.blackbasta	2
win.ryuk	42
elf.hive	3
win.darkside	15
win.medusalocker	8
elf.whiterabbit	1
win.phobos	4
win.hermes	4
win.snatch	1
win.wastedlocker	13
win.cryptowall	60
win.stop	11
win.egregor	3
elf.blacksuit	1
elf.bianlian	1
win.agendacrypt	2
win.bianlian	15
win.blackbyte	9
win.wannacryptor	11
win.babuk	12
win.ragnarlocker	11
win.gocryptolocker	1
elf.lockbit	6
win.8base	2
Total	698

2.2.2 Caracterización por medio de análisis estático

Para comprender en profundidad las muestras, se realiza un análisis estático, es decir, se revisan los archivos sin ejecutarlos. Esta metodología resulta especialmente relevante durante la investigación, ya que garantiza que no ejecutemos accidentalmente código potencialmente peligroso (Jelodar et al., 2025).

El análisis estático permite obtener información relevante, como los valores hash criptográficos (Bharti et al., 2023), la estructura interna del archivo, las cadenas de texto presentes, los indicadores de compromiso (IoC), las funciones y los elementos requeridos de otros programas y bibliotecas (imports y exports), el nivel de entropía y los patrones de ofuscación empleados para ocultar su funcionalidad (Naik, Jenkins, Savage, et al., 2020). Estos datos resultan fundamentales para conformar la colección de información que utilizará el sistema RAG.

2.2.2.1 Construcción del script para la caracterización de las muestras

Al aplicar los criterios de reducción de muestras de malware, se obtienen 698 muestras. Se crea un script en Python llamado `static_extractor.py`, con el cual se obtienen automáticamente las características fijas o estáticas de las muestras. Garantizando su funcionamiento en ambientes seguros, sin ejecutar las muestras, para que el análisis no resulte peligroso.

El script analiza muchos tipos de archivos: archivos ejecutables de Windows (PE), archivos de Linux (ELF), documentos de PDF y archivos comprimidos. La forma en que extrae la información es en un archivo JSON.

Este script nos permite clasificar mediante análisis de firmas binarias el tipo de archivo:

- PE (Windows)
- ELF (Linux)

El script determina cuánta entropía tiene la muestra, junto con sus distintas partes. Con este número se puede determinar si se han utilizado formas de esconder el código o de ofuscación porque si la entropía es alta, normalmente significa que el archivo contiene información que ha sido encriptada o comprimida.

Lo que más destaca de este programa es que es capaz de encontrar texto oculto dentro de las muestras. Encuentra tanto texto ASCII normal como texto en formato UTF-16. Además, de esos archivos saca pistas de que algo es sospechoso (IoC): las direcciones de páginas de internet (URLs), los nombres de esas páginas (dominios), y las direcciones IP. Estos datos son muy importantes para hacer reglas de YARA, porque son ejemplos específicos que se pueden usar directamente para reconocer programas peligrosos (Wang et al., 2025).

Cuando se trata de archivos PE (ejecutables de Windows), el script examina a fondo su interior, y para ello usa programas específicos en Linux como *pefile*. De estos archivos, el programa saca las importaciones, es decir, qué funciones de otros programas utiliza, y también las exportaciones. Analiza también las secciones de los archivos (cuánto miden y su grado de aleatoriedad) y determina desde dónde empieza a ejecutarse el programa. Con todos estos datos, podemos hacernos una idea de lo que el ransomware podría hacer y reconocer características típicas de ciertas familias de malware.

El script analiza el código a nivel de instrucciones mediante la generación de un histograma de opcodes, utilizando la librería Capstone en Linux, que consiste en cuántas veces aparece cada código de operación o contabilizar la frecuencia de cada instrucción. Una representación así ayuda a encontrar la forma en que está hecho el código, encontrar patrones en este e identificar familias de malware.

Además, la librería LIEF en Linux se emplea para conseguir más detalles sobre cómo está organizada la muestra del malware, como el formato del archivo, las secciones y los imports de librerías que utiliza.

El script tiene instrucciones para encontrar indicios de que se ha usado algún tipo de programa para "empaquetar" el código, por ejemplo, si aparecen firmas identificativas como las de UPX, o si ciertas partes del código tienen un alto grado de entropía. Estas pistas son relevantes porque nos ayudan a reconocer programas maliciosos que están ocultando su verdadero comportamiento.

Para terminar, el script crea un archivo JSON para cada muestra, y este archivo contiene todas las características que hemos extraído de ella. Así, podemos crear una colección ordenada de datos, que luego se usaran para búsquedas en el sistema RAG.

Este nuevo repositorio de archivos JSON llamado *StaticAnalysis* que crea el script son el fundamento del conjunto de conocimiento que usa el sistema RAG. Estos simplifican el indexar en bases de datos vectoriales, recuperar información rápidamente y formar reglas YARA teniendo en cuenta el contexto debido a la información estructurada y con contexto.

Figura 3.

Archivo .JSON generado por el script static_extractor.py después de realizar el análisis estático de una muestra de ransomware

```
518544e56e8ccee401ffa1b0a01a10ce23e49ec21ec441c6c7c3951b01c1b19c.static.json
{
  "ransomware": "win.8base",
  "file_path": "win.8base/518544e56e8ccee401ffa1b0a01a10ce23e49ec21ec441c6c7c3951b01c1b19c",
  "file_name": "518544e56e8ccee401ffa1b0a01a10ce23e49ec21ec441c6c7c3951b01c1b19c",
  "collected_at": "2026-03-09T17:26:01.222737Z",
  "md5": "2809e15a3a54484e042fe65fffd17409",
  "sha1": "4a8f0331abaf8f629b3c8220f0d55339cfa30223",
  "sha256": "518544e56e8ccee401ffa1b0a01a10ce23e49ec21ec441c6c7c3951b01c1b19c",
  "size": 287744,
  "file_type_detected": "PE WINDOWS",
  "file_entropy": 6.485421603999748,
  "ascii_strings_count": 2295,
  "utf16_strings_count": 35,
  "ascii_strings": [
    "This program cannot be run in DOS mode.",
    "URich",
    ".text",
    ".data",
    ".rsrc",
    "@.reloc",
    "bad allocation",
    "string too long",
    "invalid string position",
    "Unknown exception",
    "LC TIME",
    "LC NUMERIC",
    "LC MONETARY",
    "LC CTYPER",
    "LC COLLATE",
    "LC ALL",
    "\\"#$%&'()*+,-./:0123456789;=<?@ABCDEFGHIJKLMNopqrstuvwxyz[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~",
    "\\"#$%&'()*+,-./:0123456789;=<?@abcdefghijklmnopqrstuvwxyz[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~",
    "\\"#$%&'()*+,-./:0123456789;=<?@ABCDEFGHIJKLMNopqrstuvwxyz[\\]^_`ABCDEFGHIJKLMNopqrstuvwxyz{|}~",
    "CorExitProcess",
    "runtime error ",
    "TLS error",
    "SING error",
    "DOMAIN error",
    "R6034",
    "An application has made an attempt to load the C runtime library incorrectly.",
    "Please contact the application's support team for more information.",
    "R6033",
    "Attempt to use MSIL code from this assembly during native code initialization",
    "This indicates a bug in your application. It is most likely the result of calling an MSIL-compiled (/clr) function from a native constructor or from DllMain.",
    "R6032",
    "- not enough space for locale information",
  ]
}
```

2.3 CAPÍTULO 3: Diseño e implementación del sistema RAG para la generación de reglas YARA

Para avanzar con el tercer objetivo propuesto, se diseña un sistema que usa Retrieval-Augmented Generation (RAG), que ayudará a crear reglas YARA para identificar ejemplos de ransomware. Esta forma de hacerlo es porque los modelos de lenguaje tienen problemas cuando se usan solos, sobre todo en ámbitos muy técnicos como la seguridad informática, donde es importantísimo ser exactos, tener en cuenta el contexto y que la información esté al día (Zhang et al., 2025).

En vez de usar un modelo de lenguaje directamente, un sistema RAG junta la habilidad de un Large Language Model (LLM) para inventar cosas con un sistema para encontrar información en una colección de conocimientos contextualizados. En este proyecto, esa colección de conocimientos está formada por datos organizados obtenidos de muestras de ransomware para ofrecer un contexto adecuado al momento de generar reglas YARA.

Por lo tanto, este capítulo explica cómo funciona RAG, sus partes más importantes y por qué elegimos las herramientas concretas para hacer este sistema.

2.3.1 Fundamentos del RAG

La idea de Recuperación y Generación Aumentada es una forma de combinar dos cosas que se complementan: encontrar información importante en un conjunto de documentos y luego usar un modelo de lenguaje para responder a preguntas. Lo que se busca es que las respuestas sean mejores, más correctas y se entiendan mejor por el contexto en el que están (Lewis et al., 2021).

En general, un sistema RAG funciona así: primero se crea una colección de conocimiento a partir de documentos, informes, información obtenida de ejemplos o cualquier otra fuente relevante. Después, esa información se convierte a una forma de números (llamados vectores) usando un modelo de 'embeddings'. Estos números se guardan en una base de datos de vectores para que luego sea fácil encontrar cosas que se relacionen por su significado. Cuando alguien pregunta algo, la pregunta también se convierte en un vector. El sistema busca los fragmentos de texto más parecidos en significado. Y al final, esos fragmentos se añaden a la instrucción que se le da al modelo de lenguaje para que produzca una respuesta basada en lo que encontró.

Lo bueno de usar este sistema es que el modelo ya no tiene que depender solamente de todo lo que ya sabía. Así, se puede trabajar con información limitada, que podemos revisar para que sea correcta, y que está relacionada con el tema específico que se trata. Este proyecto es muy importante para crear reglas YARA, porque el modelo puede usar descripciones técnicas, pistas que se encontraron y formas típicas de los malware que se usan para afectar sistemas informáticos.

2.3.2 Componentes del RAG propuesto

Para este estudio, el sistema RAG se diseñó usando varios componentes.

Base de conocimiento:

La base de conocimiento es la información que se recopiló y organizó en las etapas previas del proyecto. Contiene las características del análisis estático de los malware que se extrajeron usando el script en Python planteado en el capítulo anterior. Esta base tiene claves únicas de los archivos, texto dentro de ellos, llamadas a funciones, partes del código, un cálculo de aleatoriedad, detalles de cómo se identifican como peligrosos y otros detalles importantes que ayudan a generar información.

Proceso de fragmentación o chunking:

Como los documentos de las características de cada malware pueden ser muy extensos en cuanto a información extraída de las muestras de malware, fue necesario separarlos en porciones menores, llamados chunks. Dividirlos así hace que encontrar información relacionada sea más rápido y evita que el modelo se vea desbordado por información que no es importante. Cada parte es una unidad de información que se puede encontrar y usar según lo que se pregunta.

Modelo de embeddings:

Una vez dividida la información, cada porción de texto tiene que convertirse en una forma numérica que conserve su significado. Estas formas numéricas, llamadas embeddings, permiten comparar si los documentos y las preguntas tienen ideas parecidas. En el sistema RAG el modelo de embeddings es importantísimo, porque de él depende lo bien que se encuentre la información.

Base de datos vectorial:

Los embeddings que se crearon se almacenan en una base de datos vectorial. Esta permite buscar información parecida en significado. Una base de datos vectorial no sólo busca coincidencias exactas de texto como harían las bases de datos que normalmente se usan, sino que busca conceptos que estén cerca uno del otro en términos de las representaciones numéricas. Esto es muy útil en seguridad informática, porque la misma técnica o comportamiento de un programa malicioso puede describirse de muchas formas diferentes.

Modelo de lenguaje:

El LLM es el componente del sistema que realmente genera respuestas. Recibe tanto la pregunta del usuario como la información que se encontró en la base de datos vectorial y, con eso, produce una respuesta que sirva para algo y que sea correcta desde el punto de vista técnico. En este caso, se espera que la respuesta proporcione información útil para escribir reglas YARA.

Interfaz:

Por último, el sistema necesita una forma fácil de subir documentos, buscar en la colección de archivos y hablar con el modelo. Esta parte integra las partes anteriores en un proceso que se puede usar y probar fácilmente.

2.3.3 Componentes tecnológicos de la implementación

Ollama como motor de inferencia de LLM:

Ollama² es lo que se usó para que el modelo de lenguaje funcione en la propia computadora. Esto es porque con Ollama se pueden instalar modelos que ocupan menos espacio (modelos cuantizados) bastante fácilmente en un servidor Linux, y así no se necesitan servicios en línea. Gracias a Ollama, usar y controlar el modelo DeepSeek-r1:8b fue mucho más fácil, y todo el proceso de cálculo se quedó en un lugar seguro.

Al elegir Ollama para este proyecto se obtienen muchas ventajas. Primero, se pudo hacer que el modelo de lenguaje funcione en la propia computadora, lo que ayuda a que los datos sean privados y es más fácil demostrar que los resultados son correctos. Segundo, poner el modelo y usarlo fue mucho más sencillo, bajando la dificultad de ponerlo en marcha. Por último, gracias a que se puede usar e integrar con programas como Open WebUI, el elemento del sistema que se encarga de crear texto se conectó muy bien.

Modelo de embeddings:

Se utilizó un modelo de "embeddings" para transformar tanto los documentos como las partes de los documentos que tenemos (el "corpus") en vectores que representan su significado. Este componente es importantísimo para un sistema RAG, porque es el encargado de decidir cómo

² <https://ollama.com/>

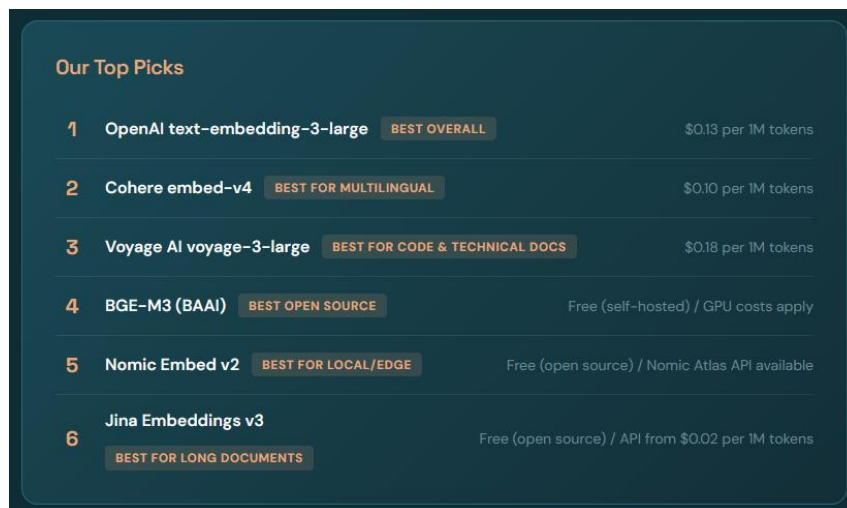
se entiende y se sitúa el contenido en un espacio de números (un espacio vectorial), y de eso depende mucho que encontremos la información correcta.

Este proceso de obtener los vectores se aplicó a los documentos que se hicieron con la información del proyecto: descripciones de programa de muestras de ransomware, elementos que nos dan pistas de que un programa es peligroso (indicadores de compromiso), las características que se encontraron analizando los programas detenidamente (análisis estático) y otras cosas importantes para crear las reglas para YARA. En otras palabras, cada porción de texto se convirtió en una lista de números que explica de qué trata, de forma que luego se pueda comparar con lo que el usuario pregunta.

El modelo de embedding seleccionado para este proyecto es BGE-M3³ el cual es uno de los modelos open source más usado para RAG.

Figura 4.

Top modelos de embeddings: BG3-M3 está catalogado como el mejor modelo embedding Open Source en varios estudios de 2026. <https://pecollective.com/tools/best-embedding-models/>



Our Top Picks		
1	OpenAI text-embedding-3-large	BEST OVERALL \$0.13 per 1M tokens
2	Cohere embed-v4	BEST FOR MULTILINGUAL \$0.10 per 1M tokens
3	Voyage AI voyage-3-large	BEST FOR CODE & TECHNICAL DOCS \$0.18 per 1M tokens
4	BGE-M3 (BAAI)	BEST OPEN SOURCE Free (self-hosted) / GPU costs apply
5	Nomic Embed v2	BEST FOR LOCAL/EDGE Free (open source) / Nomic Atlas API available
6	Jina Embeddings v3	BEST FOR LONG DOCUMENTS Free (open source) / API from \$0.02 per 1M tokens

Base de datos vectorial ChromaDB:

Para guardar y encontrar información relevante, se usa ChromaDB, que es una base de datos de vectores buena para el proyecto. Lo que hace dentro del programa es guardar las

³ <https://huggingface.co/BAAI/bge-m3>

representaciones de significado de los textos, y buscar entre ellas cuando preguntas algo, encontrando lo que es más parecido a tu pregunta.

Se opta por ChromaDB porque es fácil de añadir a lo que se propone, además que funciona bien con el sistema RAG. En este trabajo, gracias a ChromaDB se puede organizar los trozos de texto creados a partir de todos los documentos.

Este elemento es importantísimo porque el modelo de lenguaje no busca en todos los documentos, solo en los trozos que ChromaDB dice que son los más importantes para cada pregunta.

Open WebUI como orquestador:

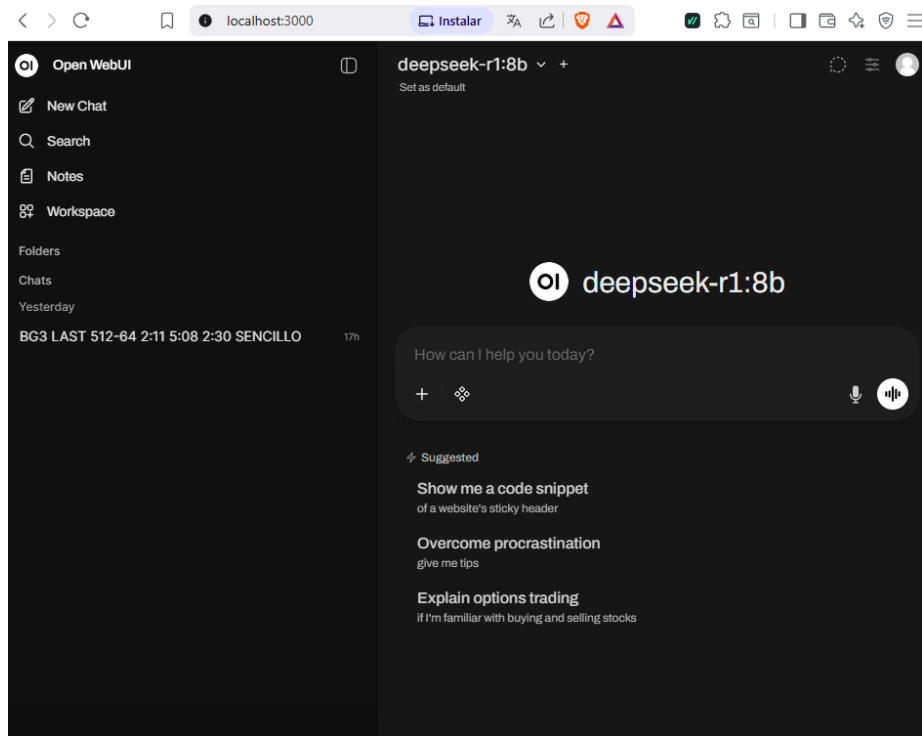
Se usa Open WebUI⁴ para interactuar con el sistema RAG. De hecho, Open WebUI fue la forma de preguntar por información almacenada, darle órdenes al modelo y ver las respuestas que daba. Gracias a esta plataforma, se tuvo todo el proceso de trabajo del sistema RAG en un mismo sitio, sin necesidad de crear una interfaz de usuario desde el principio.

En el prototipo que se realiza, se usa Open WebUI tanto para escribir las preguntas como para probar si el sistema es capaz de responder correctamente basándose en los datos sobre ransomware que tiene ordenados. Además, hizo más fácil organizar los documentos y coordinar el trabajo entre el modelo de lenguaje y la base de datos vectorial.

Figura 5.

Open WebUI funciona como orquestador que ayuda a integrar la base de datos vectorial, el modelo de embedding y el LLM para recuperación de datos.

⁴ <https://openwebui.com/>



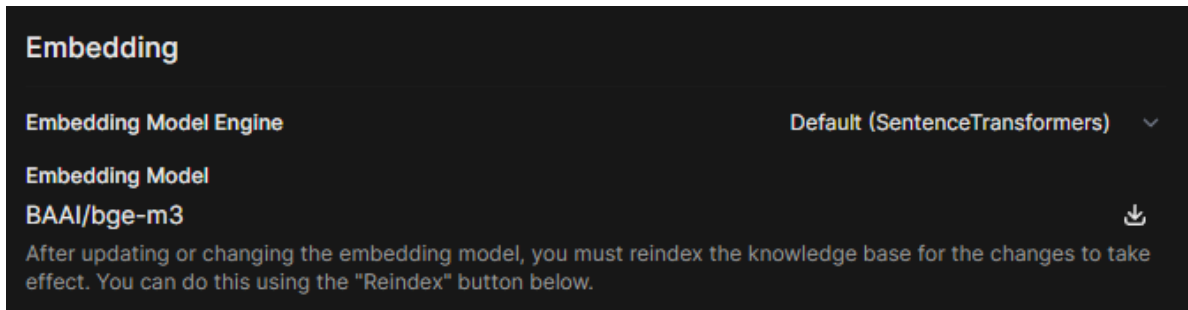
2.3.4 Ciclo de funcionamiento del RAG

En la primera fase del proyecto, se recopiló y ordenó toda la información que se había conseguido antes. Esta información venía de los archivos exportables JSON de los informes que había creado el programa de análisis estático. De estos lugares se obtuvieron descripciones de distintas familias de ransomware, sus códigos de referencia (hashes), cadenas de texto, partes internas, qué funciones usaban, su nivel de aleatoriedad, gráficos de instrucciones del procesador y elementos que pueden servir para identificarlos.

Después de reunir todo esto, se dio forma a los elementos de manera que se pudieran leer fácilmente y de forma uniforme, preparándolos para ser indexados. Para esto se procede a convertir los archivos JSON en .md debido a que es mucho mejor para digerir en el proceso de embedding e indexado en la base de datos vectorial.

Figura 6.

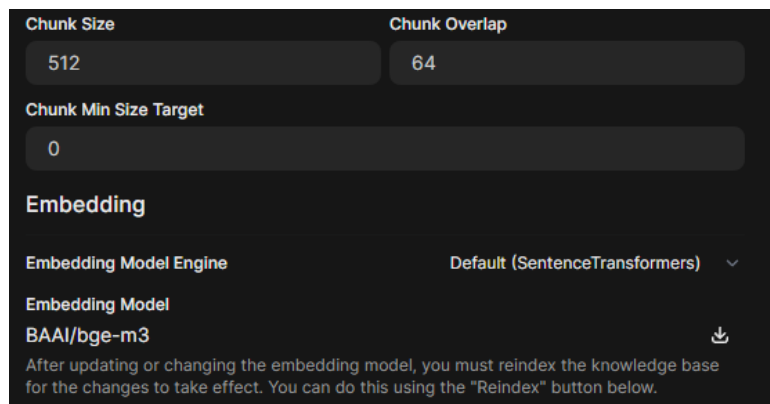
Modelo de embeddings usado BGE-M3 en OpenWebUI.



Dividir los documentos así, además, disminuye las probabilidades de obtener documentos demasiado largos o que no lleguen al punto, haciendo que la búsqueda sea más específica. Para este trabajo de investigación, cada una de esas partes podía tener datos de un tipo particular de ransomware, una serie de señales que lo identifican, o una descripción de cómo está hecho que viene del análisis estático realizado.

Figura 7.

Configuración proceso de fragmentación chunk dentro de Open WebUI



El modelo de embeddings convirtió cada parte del texto en una forma de vector. Esto significa que el contenido de las palabras se ubicó en un ámbito matemático: cuanto más cerca estén los vectores, más parecido será el significado de lo que dicen.

Ahora, el programa ya no necesita encontrar exactamente las mismas palabras, sino que trabaja con el sentido que tienen. Esto es muy importante para la seguridad informática, porque una acción peligrosa puede describirse usando palabras diferentes, pero con la misma idea de fondo.

Después de tener los vectores, se guardaron en ChromaDB con los pedazos originales de texto y sus datos relacionados. Al indexarlos, se creó una colección de vectores que se pueden buscar, y en cada elemento de esta colección tanto el número que lo representa, como el texto en sí, se conservan.

Por lo tanto, cuando alguien pregunta algo, el sistema puede encontrar los vectores más parecidos, también puede encontrar los propios textos para usarlos como información de apoyo para el modelo que genera las respuestas.

Figura 8.

Indexación en ChromaDB del repositorio del análisis estático realizado a las muestras de ransomware

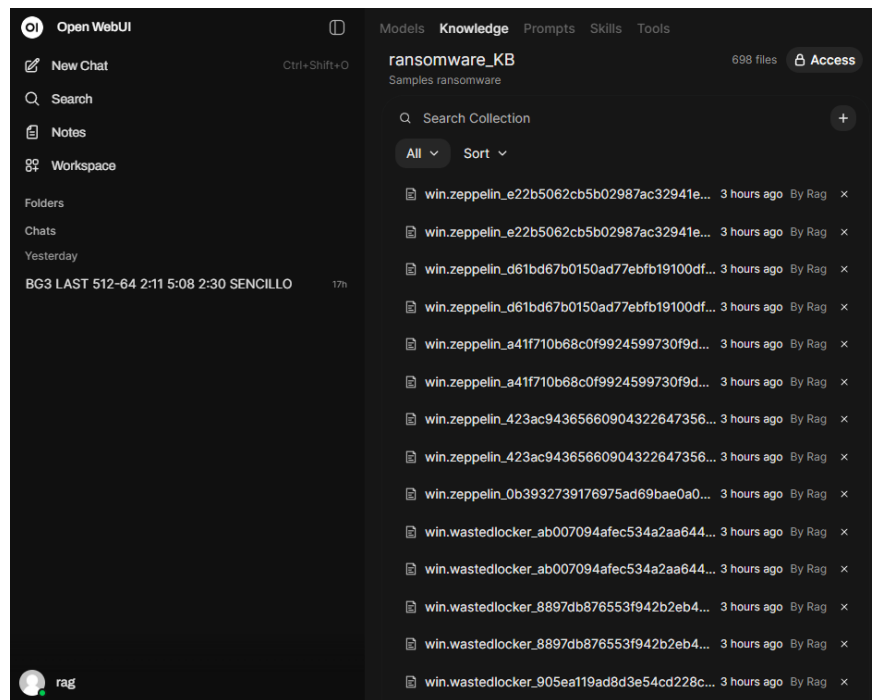
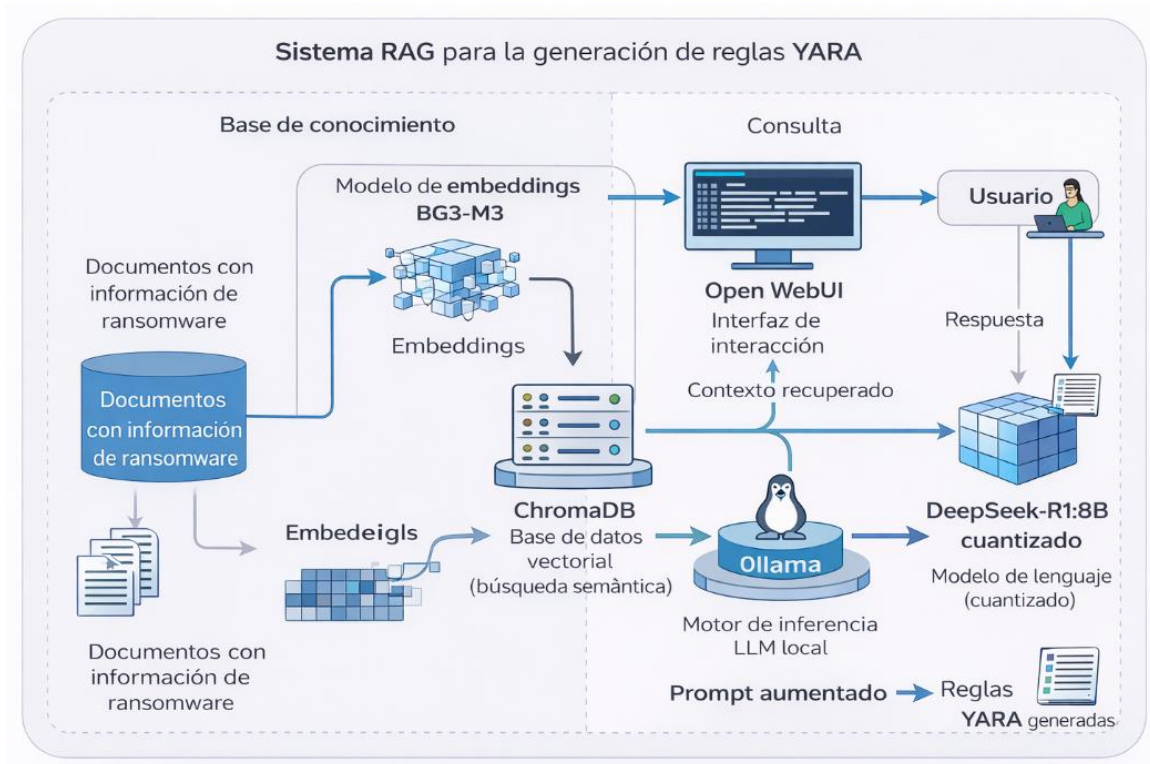


Figura 9.

RAG con todos sus componentes vinculados. Nota: Imagen generada con ChatGPT (GPT Image, OpenAI, 2026)



2.3.5 DeepSeek como LLM para el proyecto y selección del modelo DeepSeek 8B cuantizado

En un mundo lleno de modelos de lenguaje (LLM), se decide elegir DeepSeek⁵, debido a que tiene novedades técnicas, facilidad y libre de acceso.

Primero, es gratis. Muchas otras opciones te ponen límites en el número de preguntas que puedes hacer al día, o te dan menos herramientas si usas la versión gratuita.

Segundo, tiene una memoria enorme de 1 millón de tokens. Con DeepSeek puedes analizar de golpe textos tan largos. Esto es perfecto para estudiar trabajos muy largos, investigaciones o proyectos complicados.

⁵ <https://www.deepseek.com/en/>

Tercero, razona de forma profunda y te enseña cómo llega a sus ideas. En vez de darte solamente el resultado, DeepSeek te dice por qué piensa lo que dice. Esto es muy útil cuando tienes que programar, hacer cuentas o usar la lógica, sobre todo porque puedes ver de dónde sale la solución.

Cuarto, entiende muchas formas de información: texto, imágenes, archivos PDF, presentaciones de PowerPoint, hojas de cálculo de Excel, documentos de Word y fotos. Puedes subir lo que necesites y sacar información de ello, sin tener que usar otros programas.

Quinto, es de código abierto y hace lo posible para que todo se vea. A diferencia de modelos como GPT-4 o Claude que son secretos, DeepSeek ha abierto versiones llamadas “open-weight” (DeepSeek-V2 y V3, por ejemplo), para que todo el mundo pueda investigar, comprobar y mejorar su tecnología. Esto ayuda a confiar más en él y a que todos puedan innovar.

Sexto, su integración con Ollama y Open WebUI es muy importante para la construcción de un entorno reproducible, privado y funcional para el sistema RAG propuesto.

Se decide usar el modelo DeepSeek-R1:8B cuantizado para la parte del sistema RAG que genera la recuperación del texto, sobre todo porque es eficiente en cuanto a los cálculos que necesita, funciona bien en tareas técnicas y es compatible con el hardware que tenemos.

Primero, los modelos de lenguaje de tamaño medio, como los de 8 mil millones de parámetros, son suficientemente buenos para entender y crear contenido en áreas especializadas. Para este estudio, el modelo tiene que comprender información técnica sobre programas de rescate, incluyendo indicadores de compromiso, cómo están contruidos los archivos, qué funciones usan, cadenas de texto y otros aspectos importantes para crear normas YARA. DeepSeek-R1:8B es suficientemente bueno para este tipo de cosas, especialmente si se usa dentro de una estructura RAG que le da más información.

En segundo lugar, se tuvo muy en cuenta si el modelo era factible de usar con el equipo computacional disponible. A diferencia de los modelos más grandes, uno de 8 mil millones de parámetros necesita mucha menos memoria y potencia de procesamiento, facilitando que funcione en ordenadores sin una tarjeta gráfica de mucha capacidad. Esto es importante en este

proyecto dado que el servidor que estamos usando tiene pocos recursos y hay que usar la memoria de la mejor manera.

Usar el modelo en formato cuantizado ayuda aún más a reducir la cantidad de recursos que necesita, sin que las respuestas se vean afectadas. Así, se consigue un buen punto entre calidad y rapidez, perfecto para un experimento que podamos repetir.

Otro aspecto clave para elegir DeepSeek-R1:8B es que se adapta bien al enfoque RAG. En este tipo de sistema, el modelo no solo usa lo que ya sabe, sino que recibe información importante que se busca en una base de datos de vectores. Por eso, no es necesario usar modelos muy grandes, ya que la información extra que se le da compensa cualquier falta de conocimiento que pueda tener. De hecho, DeepSeek-R1:8B es suficiente para entender la información que se busca y generar respuestas lógicas y útiles desde el punto de vista técnico.

Además, el modelo funciona bien con herramientas como Ollama para ponerlo en marcha en el ordenador, lo que permite que se integre fácilmente en el sistema que proponemos. Esto facilita crear un ambiente controlado para hacer pruebas, cambiar ajustes y ver los resultados sin necesidad de usar servicios de fuera.

En resumen, la elección de DeepSeek-R1:8B cuantizado es el resultado de buscar un equilibrio entre su capacidad para crear texto, lo poco que consume y su idoneidad para el enfoque RAG. Gracias a esta elección podemos tener un sistema que funcione con el equipo que tenemos, y que al mismo tiempo sea suficientemente bueno para los objetivos del estudio.

2.3.6 Infraestructura computacional utilizada

El sistema se instaló y se probó en un equipo local. Sus componentes internos hicieron posible utilizar el modelo de lenguaje, crear las representaciones de significado (embeddings) y ejecutar todo el sistema RAG al mismo tiempo. El equipo tiene las siguientes especificaciones:

Procesador: AMD Ryzen 7 7435HS

Memoria RAM: 16 GB

Unidad de procesamiento gráfico (GPU): NVIDIA GeForce RTX 4050

Almacenamiento: 500 GB SSD NVMe

El procesador Ryzen 7 7435HS tiene muchos núcleos de procesamiento, lo que facilita hacer muchas cosas al mismo tiempo: dividir los documentos, construir los embeddings y añadir la información a la base de datos vectorial. Los 16 GB de memoria RAM son algo limitados en comparación con ordenadores con más RAM, pero al usar modelos de lenguaje más pequeños, el sistema funciona bien sin perder nada.

Tener una tarjeta gráfica NVIDIA GeForce RTX 4050 es importante porque acelera el proceso de obtener resultados del modelo (inferencia), e incluso podría acelerar la creación de embeddings según la configuración. El sistema puede funcionar sin la tarjeta gráfica (con el procesador solamente), pero si se usa la tarjeta gráfica, el modelo responde más rápido y funciona mejor en general.

En cuanto al almacenamiento, el disco SSD NVMe de 500 GB es muy rápido al leer y escribir información, algo clave para cargar los modelos, acceder a la base de datos vectorial y manejar muchos documentos.

En resumen, esta configuración permitió crear un sistema RAG que funciona. Demuestra que se pueden usar modelos de lenguaje y sistemas de búsqueda por significado en ordenadores personales que tengan suficientes recursos, sin necesidad de un equipo muy sofisticado y caro.

Figura 10.

Desempeño de la GPU NVIDIA GeForce RTX 4050 en las fases indexación, recuperación y generación del RAG.

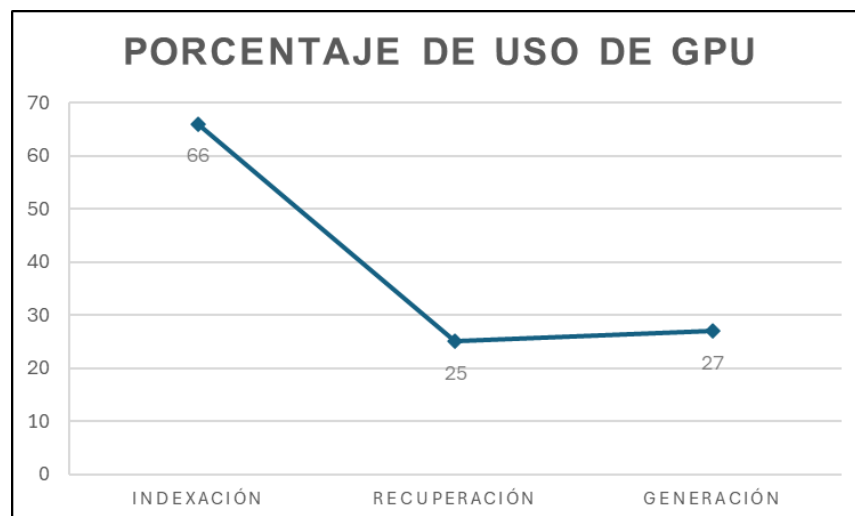


Figura 11.

Desempeño de la RAM 16 GB en las fases indexación, recuperación y generación del RAG.

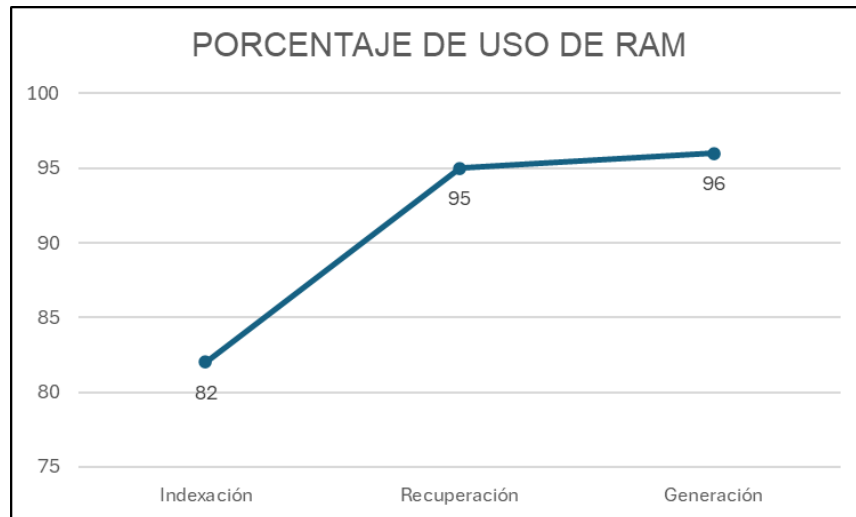
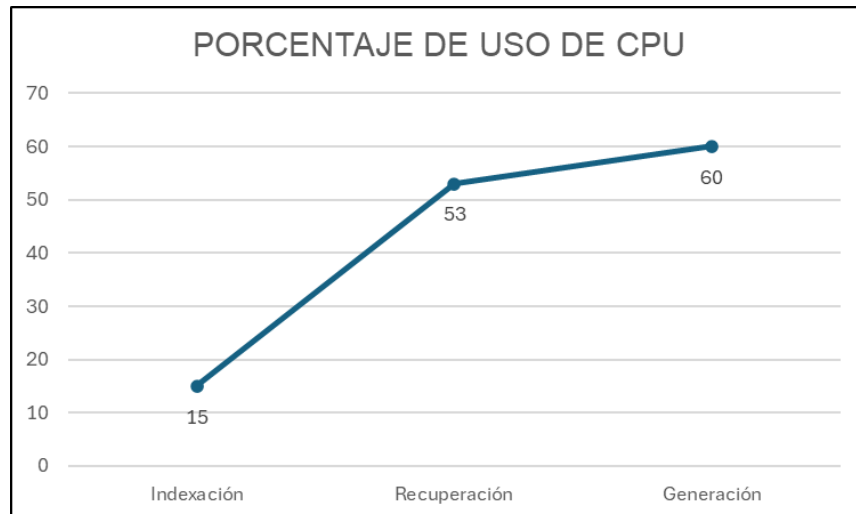


Figura 11.

Desempeño de la CPU Ryzen 7 7435HS en las fases indexación, recuperación y generación del RAG.



2.4 CAPÍTULO 4: Diseño del prompt para la generación de reglas YARA en el sistema RAG

Para llevar a cabo el cuarto objetivo planteado, se trata de crear reglas YARA automáticamente usando el sistema RAG que ya se ha hecho, además se añade una forma de que esas reglas vayan mejorando de forma iterativa llamada self-reflection (B. Li & Zhao, 2025). Este enfoque busca no solo generar reglas iniciales, sino también evaluarlas críticamente y refinarlas para mejorar su calidad, reduciendo falsos positivos y fortaleciendo su capacidad de detección.

Se realiza este enfoque debido a que la construcción de reglas YARA directamente en un LLM puede presentar limitaciones y es posible que tengan problemas: puede incluir formas muy generales de un malware y patrones comunes los cuales no serán suficientemente específicas generando posiblemente falsos positivos. Por eso se ha diseñado un proceso de tres pasos para ir haciendo las reglas mejores y mucho más específicas.

El proceso propuesto se estructura en tres fases secuenciales:

- Generación inicial de la regla YARA (Prompt inicial)
- Evaluación crítica de la regla generada (Self-reflection)
- Refinamiento de la regla YARA (Prompt de mejora)

Este flujo permite simular un proceso similar al de un analista humano, donde una primera versión es revisada y posteriormente optimizada.

2.4.1 Prompt inicial

La primera etapa consiste en la generación de una regla YARA a partir del contexto recuperado por el sistema RAG. En esta fase, el modelo de lenguaje recibe un prompt estructurado que define claramente las restricciones y elementos que deben incluirse en la regla (Leviathan et al., 2025).

A continuación, el prompt inicial:

“Necesito que ejecute el siguiente prompt según cada paso de manera secuencial:

Como analista sénior de código de malware, use EXCLUSIVAMENTE la información de todos los documentos recuperados y diseñe una regla YARA efectiva incluyendo en la regla generada lo siguiente:

- strings mas relevantes*
- strings .dll*
- strings .exe*
- hashes*
- tipo de archivo*
- IOC*
- APIs*
- URL*
- Dominios*
- Entropia*

Resultado en español

Resultado en formato de regla YARA

No inventar datos o poner datos de ejemplo en la regla generada

No incluir el campo date

Sin explicación, sin limitaciones”

Este prompt tiene ciertas condiciones que son esenciales. Para empezar, el modelo sólo puede usar la información que el sistema RAG le dé, con el fin de no generar información inventada. Luego, se le dice exactamente qué tiene que formar parte de la regla, de modo que contenga variables importantes que se pueden analizar gracias al análisis estático.

Así, de esta forma, se obtiene un primer borrador de la regla YARA. Este borrador demuestra que el modelo puede combinar diferentes tipos de pruebas dentro de una forma organizada.

2.4.2 Evaluación mediante self-reflection

Una vez generada la regla inicial, se aplica un proceso de self-reflection (B. Li & Zhao, 2025), en el cual el modelo evalúa críticamente su propia salida. Esta técnica ha sido utilizada en modelos de lenguaje para mejorar la calidad de las respuestas mediante iteraciones internas.

El prompt utilizado para esta fase es el siguiente:

“Analiza críticamente la regla YARA generada. Identifica cadenas demasiado genéricas, riesgos de falsos positivos, condiciones débiles o excesivamente restrictivas, e indica qué mejoras técnicas deberían aplicarse de forma resumida.”

Durante esta etapa, el modelo no genera una nueva regla, sino que produce un análisis crítico que puede incluir:

- Identificación de strings demasiado comunes
- Detección de condiciones poco discriminantes
- Evaluación del riesgo de falsos positivos
- Identificación de ausencia de condiciones relevantes
- Problemas de balance entre precisión y cobertura

Este paso es fundamental, ya que permite detectar debilidades que no son evidentes en la generación inicial. En términos metodológicos, introduce un nivel de validación intermedia que mejora la calidad del resultado final.

El uso de self-reflection en la generación de reglas YARA aporta múltiples beneficios debido a que estas pueden resultar ser mejores porque podemos encontrar fallos y puntos débiles en ellas. Se producen menos falsos positivos, sobre todo si quitamos patrones demasiado amplios y genéricos. Técnicamente, la búsqueda se vuelve más sólida al reforzar los criterios que se emplean para encontrar patrones. Además, el proceso de crear las reglas se parece más a como un analista humano revisaría y mejoraría continuamente su trabajo. Y finalmente, se aprovecha mucho mejor el contexto que proporciona el RAG, comprobando si la información que se busca se está aplicando correctamente. Esta manera de hacerlo es mucho mejor que generar las reglas YARA con tan solo un prompt.

2.4.3 Refinamiento de la regla YARA

La tercera etapa consiste en la generación de una versión mejorada de la regla YARA, incorporando las observaciones identificadas en la fase de self-reflection.

El prompt utilizado es el siguiente:

“Reescribe la regla YARA anterior incorporando las mejoras identificadas en la revisión. Mantén coherencia con la evidencia original y optimiza el balance entre precisión y cobertura. No incluir el campo date.”

En esta fase, el modelo genera una nueva versión de la regla que busca:

- Eliminar o ajustar strings genéricos
- Mejorar las condiciones lógicas
- Incorporar indicadores más discriminantes
- Balancear la detección para reducir falsos positivos
- Mantener coherencia con los datos originales del análisis

El resultado es una regla más robusta, que refleja no solo la información disponible, sino también un proceso de validación y mejora iterativa. Adicionalmente se menciona que el tiempo promedio de la construcción de una regla YARA desde el prompt inicial, self-reflection y refinamiento es de 10 minutos con la infraestructura computacional de este proyecto.

La integración de este mecanismo dentro del sistema RAG fortalece la propuesta del proyecto, al combinar recuperación de conocimiento, generación automática y validación crítica en un mismo flujo. De esta manera, se logra un sistema más robusto, capaz de producir reglas YARA más precisas y útiles en el contexto de la detección de ransomware (Zhang et al., 2025).

2.5 CAPÍTULO 5: Métricas para la evaluación de las reglas YARA RAG y LLM sin RAG

Para llevar a cabo el quinto objetivo específico, se comprueba la efectividad de las reglas YARA generadas por el sistema RAG propuesto. Queremos ver cuánta capacidad tienen para detectar archivos de tipo ransomware. Para esto comparamos dos formas de hacer las reglas: una que

usa un modelo de lenguaje dentro de una estructura RAG (que sí tiene acceso a información ya encontrada), y otra que usa el modelo de lenguaje directamente.

En el tema de la seguridad informática, evaluar las reglas de YARA es un paso importante y fundamental para que estas no identifiquen como peligrosos archivos que en realidad no lo son.

2.5.1 Enfoques para evaluar

Para el desarrollo experimental se consideran dos enfoques principales:

- LLM sin RAG: generación directa de reglas YARA utilizando el modelo de lenguaje sin acceso a información contextual específica (Al-Karaki et al., 2024).
- LLM con RAG: generación de reglas YARA utilizando el sistema propuesto, donde el modelo recibe información relevante recuperada desde la base de conocimiento.

La comparación entre ambos enfoques permite evaluar el impacto del uso de recuperación de información en la calidad de las reglas generadas.

2.5.2 Métricas de evaluación

2.5.2.1 Tasa de verdaderos positivos (TPR / Recall)

El recall mide la capacidad de la regla para detectar correctamente muestras maliciosas:

$$Recall = \frac{TP}{TP + FN}$$

Donde:

TP: verdaderos positivos (malware detectado correctamente)

FN: falsos negativos (malware no detectado)

Una regla con alto recall es capaz de detectar la mayoría de las amenazas.

2.5.2.2 Precisión

La precisión mide la proporción de detecciones correctas sobre el total de detecciones:

$$Precision = \frac{TP}{TP + FP}$$

Donde:

FP: falsos positivos (archivos benignos detectados como malware)

Esta métrica es clave para evitar alertas innecesarias en entornos reales.

2.5.2.3 F1 - Score

El F1-score combina precisión y recall en una sola métrica:

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Permite evaluar el balance entre detección y exactitud.

2.5.2.4 Tasa de falsos positivos (FPR)

La tasa de falsos positivos mide la proporción de archivos benignos incorrectamente detectados:

$$FPR = \frac{FP}{FP + TN}$$

Donde:

TN: verdaderos negativos (archivos benignos correctamente ignorados)

Una baja FPR es esencial para la aplicabilidad práctica de las reglas.

2.5.2.5 CDR (Cross Detection Rate)

Mide cuántas veces una regla diseñada para una familia detecta incorrectamente otras familias de malware (cross-detection). El valor ideal es 0%.

$$CDR = \frac{Cross\ detections}{Total\ de\ muestras} * 100$$

2.5.3 Comparación entre RAG y LLM sin RAG

Se espera que el enfoque basado en RAG presente mejoras en varios aspectos:

- Mayor precisión: debido al uso de contexto específico.
- Menor tasa de falsos positivos: al evitar patrones genéricos.
- Mejor balance entre cobertura y especificidad.
- Mayor coherencia técnica en las reglas generadas.

Por otro lado, el enfoque sin RAG puede presentar:

- Generación de reglas más genéricas.
- Mayor dependencia del conocimiento interno del modelo.
- Mayor riesgo de inconsistencias o invención de datos.

Tabla 3

Ejemplo de comparación de métricas principales entre generación de reglas YARA con RAG y sin RAG

Métrica	LLM sin RAG	LLM con RAG	Interpretación
Precisión (Precision)	Media / Variable	Alta	RAG reduce falsos positivos al usar contexto real
Recall (Detección de malware)	Media	Alta	RAG mejora la detección basada en evidencia
F1-Score	Medio	Alto	Mejor balance entre precisión y recall
Tasa de falsos positivos (FPR)	Alta	Baja	Sin RAG se usan patrones más genéricos
Tasa de falsos negativos (FNR)	Media / Alta	Baja	RAG reduce malware no detectado

2.6 CAPÍTULO 6: Pruebas y evaluación de las reglas YARA RAG y LLM sin RAG

En el desarrollo del sexto objetivo específico se realiza un análisis comparativo del desempeño de los modelos evaluados en la generación de reglas YARA. Este análisis se basa en cinco métricas fundamentales: F1-score, precisión, recall, Cross Detection Rate (CDR) y False Positive Rate (FPR), las cuales permiten evaluar de manera integral la calidad, efectividad y robustez de las reglas generadas.

Los modelos analizados corresponden a distintas estrategias de generación, incluyendo modelos de lenguaje sin RAG y el enfoque propuesto basado en RAG. A partir de los resultados obtenidos, se busca identificar fortalezas, debilidades y comportamientos característicos de cada modelo.

2.6.1 Definición de las métricas para la evaluación

Para cuantificar la efectividad de las reglas YARA generadas, se emplearon métricas de clasificación estándar que permiten evaluar la capacidad de discriminación entre archivos maliciosos y legítimos (H. Li & Li, 2024).

- **F1-Score:** Media armónica entre precisión y recall que representa el balance de éxito general del modelo.
- **Precisión:** Mide la proporción de detecciones correctas sobre el total de detecciones realizadas.
 - Alta precisión → pocas falsas alarmas
 - Baja precisión → alto número de falsos positivos
- **Recall (Sensibilidad):** Evalúa la capacidad del modelo para detectar muestras maliciosas reales.
 - Alto recall → mayor cobertura de amenazas
 - Bajo recall → muchas amenazas no detectadas

- **CDR (Cross Detection Rate):** Mide la capacidad de generalización del modelo, es decir, su habilidad para detectar variantes distintas del malware objetivo.
- **FPR (False Positive Rate):** Probabilidad de clasificar incorrectamente un archivo legítimo como malicioso. En entornos de producción, se busca que este valor tienda a cero.

2.6.2 Metodología de evaluación

Se generaron 40 reglas YARA de los modelos ChatGPT (GPT-5.3), Gemini (versión 2.5 flash), DeepSeek (versión 3.2) y 40 reglas de nuestro modelo construido RAG_YARA (DeepSeek R1:8B) para compararlas. En total, se seleccionaron 40 familias de ransomware, distribuidas en 30 muestras para Windows y 10 para Linux. Esta proporción responde a que la mayoría de las campañas de ransomware históricamente han estado dirigidas a sistemas Windows, debido a que es el sistema operativo más usado en el mundo tanto como personas como a nivel empresarial, mientras que las variantes para Linux representan una menor proporción relativa. La elección de las familias de ransomware se debe a su relevancia histórica, impacto operativo y diversidad técnica de cada familia. En el caso de Windows, se seleccionaron 30 familias ampliamente documentadas, incluyendo variantes clásicas como CryptoLocker y WannaCry, así como familias más actualizadas y activas como LockBit, Conti y BlackCat. Esta selección permite cubrir la evolución del ransomware desde sus inicios hasta las campañas más recientes. Para entornos Linux, se seleccionaron 10 familias relevantes como Akira, Babuk y DarkSide, que representan el crecimiento del ransomware en infraestructuras de servidores y entornos cloud.

Tabla 4.

Ransomware elegidos para generar las reglas YARA

Windows	Linux
win.8base.yar	elf.akira.yar
win.akira.yar	elf.babuk.yar
win.babuk.yar	elf.blackcat.yar
win.bianlian.yar	elf.clop.yar
win.blackbasta.yar	elf.darkside.yara
win.blackcat.yar	elf.hive.yar
win.blacksuit.yar	elf.lockbit.yar

win.cerber.yar	elf.ragnarlocker.yar
win.conti.yar	elf.revil.yar
win.cryptolocker.yar	elf.whiterabbit.yar
win.gandcrab.yar	
win.gocryptolocker.yar	
win.gpcode.yar	
win.hive.yar	
win.jigsaw.yar	
win.lockbit.yar	
win.makop.yar	
win.maze.yar	
win.medusa_http.yar	
win.nefilim.yar	
win.petya.yar	
win.play.yar	
win.ragnarlocker.yar	
win.revil.yar	
win.royal_ransom.yar	
win.ryuk.yar	
win.snatch.yar	
win.teslacrypt.yar	
win.wannacryptor.yar	
win.zeppelin.yar	

Para la evaluación de las reglas YARA, se estructuró el proyecto de una manera en la cual ejecutamos un script en Python donde ejecutamos la herramienta open source *yara*⁶ y de forma automatizada recorrimos los directorios comparando cada regla YARA (LLM y RAG YARA) con las muestras de malware por familia y las muestras benignas (Mansour et al., 2025).

⁶ <https://github.com/VirusTotal/yara/releases>

Figura 10.

Estructura de las carpetas del proyecto para ejecutar el script de python

```
proyecto
----ransomwareFamily
    elf.akira
    elf.babuk
    win.petya
    win.8base
    win.blackmatter
    .
    .
    .
    .
----muestras_benignas
    YARA_Rules
    ChatGPT
    Deepseek
    Gemini
    RAG_yara
```

2.6.3 Resultados globales

Estos resultados son productos de la evaluación de las reglas YARA generadas por cada modelo y el cálculo de las métricas (Naik et al., 2021) durante el proceso de identificación de las muestras de ransomware y muestras benignas mediante el uso de la herramienta *yara*.

Tabla 5.

Resultados del cálculo de las métricas para cada modelo.

model	f1	precision	recall	cdr	fpr
ChatGPT	0,058868238	0,061732018	0,083655727	0,064339468	0,0940625
DeepSeek	0,032272727	0,075	0,028968254	0,004529533	0,008125
RAG_Yara	0,01827896	0,032028684	0,015873016	0,010672331	0,026602564
Gemini	0,01044226	0,05	0,00625	0,000880958	0,0084375

2.6.4 Análisis profundo por modelo

2.6.4.1 ChatGPT

ChatGPT obtiene los mejores resultados en general usando F1-score y recall, lo que demuestra una gran capacidad para identificar muestras de ransomware.

Sin embargo, este rendimiento se logra a costa de:

- La mayor tasa de falsos positivos (FPR)
- Baja precisión relativa

Esto indica un comportamiento agresivo, donde el modelo prioriza la detección sobre la exactitud debido al incremento de los falsos positivos y la generalización de las reglas YARA generadas.

Interpretación:

Genera reglas YARA más genéricas

Mayor coincidencia de patrones → mayor cobertura

2.6.4.2 DeepSeek

DeepSeek destaca por:

Mayor precisión (0.0750)

Menor FPR (0.0081)

Esto lo posiciona como el modelo más confiable en términos de calidad de alertas. Además, indica una menor proporción de falsos positivos en sus detecciones. Esto sugiere que sus reglas son más estrictas o específicas.

Interpretación:

Reglas altamente específicas

Reduce falsos positivos

2.6.4.3 Gemini

Gemini muestra el rendimiento más bajo en prácticamente todas las métricas:

F1-score mínimo

Recall casi nulo

CDR insignificante

Interpretación:

Aunque mantiene un FPR bajo, su utilidad práctica es limitada debido a su incapacidad de detectar las muestras específicas de ransomware de cada familia.

2.6.4.4 RAG

El modelo construido RAG + DeepSeek R1:8B constituye el aporte central de la investigación. Este enfoque combina:

Fortalezas

- FPR moderado (mejor que ChatGPT)
- Mayor trazabilidad en la generación de reglas
- Comportamiento más controlado

Debilidades

- Baja precisión (0.0320)
- Bajo recall (0.0158)
- F1-score inferior a modelos base

Interpretación:

Las reglas YARA generadas no generaron un gran porcentaje de identificación.

Conclusión sobre RAG Yara

El enfoque promete para la detección de malware, pero para esto se requiere:

- Optimización de la infraestructura del RAG

2.6.5 Análisis comparativo entre los modelos

- ChatGPT → alto recall, alto FPR, alto CDR
- DeepSeek → alta precisión, bajo recall
- RAG_Yara → equilibrio bajo
- Gemini → bajo en todo

2.6.6 Ranking por métrica

A continuación, se muestra el resultado de cada modelo según su puntaje en cada métrica.

Tabla 6.

Mejores modelos por cada métrica.

Métrica	Mejor modelo
Recall (Detección)	ChatGPT
Precisión	DeepSeek
CDR	ChatGPT
Bajo FPR	DeepSeek

3. CONCLUSIONES

El presente trabajo permitió diseñar, implementar y evaluar un método completo para la generación de reglas YARA de forma automatizada. Para esto combinamos el análisis estático de muestras de ransomware, el uso de modelos de lenguaje y los sistemas de recuperación de información (RAG). Después de analizar los resultados obtenidos podemos concluir lo siguiente:

Lo primero, elegir y crear la base de datos con la información fue fundamental para el sistema. Poder usar datasets de malware organizadas por familias y especializadas nos dio información relevante y contextualizada con el tema, lo cual impacta en la calidad de las reglas YARA generadas. El repositorio de Malpedia al cual se pudo acceder contenía 21.660 archivos de 3.691 malware.

En segundo lugar, la caracterización de muestras de ransomware, las cuales fueron 698, mediante análisis estático permitió extraer atributos clave como strings, imports, secciones, entropía e indicadores de compromiso. Estos elementos constituyen la base técnica para la construcción de reglas YARA efectivas, demostrando que la automatización de este proceso es viable y útil dentro del flujo de generación.

En cuanto al diseño del sistema RAG, se comprueba que integrar la recuperación de información con modelos de lenguaje permite tener el control sobre la generación de las reglas YARA, así como su trazabilidad. A diferencia de usar un modelo de lenguaje solo por sí mismo, con un enfoque RAG se puede fundamentar las respuestas con la evidencia indexada permitiendo controlar la información inexacta o falsa.

No obstante, los resultados experimentales evidenciaron que el enfoque RAG presenta un comportamiento más conservador, priorizando la precisión sobre la cobertura. Esto se refleja en una menor tasa de falsos positivos (0,026602564), pero también en una reducción del recall (0,015873016). En contraste, modelos sin RAG como ChatGPT mostraron un comportamiento más agresivo, logrando mayores tasas de detección, aunque con un incremento significativo en falsos positivos.

Figura 11.

Variación de la métrica FPR de cada modelo



Para probar el sistema de forma práctica, empleamos métricas como el F1-score, la precisión, el recall, el CDR y el FPR para tener una idea completa de su rendimiento. Los resultados obtenidos demuestran que combinar la recuperación de información con la generación automática es una alternativa viable para la detección de malware.

El análisis comparativo entre modelos permitió identificar que modelos como ChatGPT destacaron en métricas globales como F1-score y recall, mientras que DeepSeek presentó mejores resultados en precisión y bajos falsos positivos.

La incorporación del mecanismo de self-reflection representó un aporte significativo al proceso de generación, permitiendo mejorar iterativamente la calidad de las reglas YARA. Este enfoque demostró ser útil para identificar debilidades en las reglas iniciales, reducir patrones genéricos y optimizar el balance entre precisión y cobertura.

Adicionalmente, se evidenció que es posible implementar este tipo de soluciones en entornos de hardware accesibles, utilizando modelos cuantizados y herramientas de código abierto, lo cual amplía su aplicabilidad en escenarios académicos y profesionales.

En términos generales, el proyecto demuestra que la combinación de análisis estático, modelos de lenguaje y arquitecturas RAG permite avanzar hacia la automatización inteligente en la generación de reglas YARA, aportando una solución que equilibra capacidad técnica y aplicabilidad en el área de ciberseguridad.

Uno de los principales beneficios de la automatización mediante RAG (Retrieval-Augmented Generation) es la disminución significativa del esfuerzo manual requerido en la creación de reglas YARA. Tradicionalmente, un analista de ciber seguridad debe seguir un proceso extenso que incluye analizar muestras manualmente, extraer strings relevantes, identificar APIs y librerías empleadas, correlacionar indicadores de compromiso, diseñar condiciones lógicas precisas y validar posibles falsos positivos. Este flujo de trabajo puede demandar varias horas por cada muestra o familia de malware analizada.

Con el enfoque propuesto, en cambio, el análisis estático extrae automáticamente las características relevantes de la muestra, mientras que el sistema RAG recupera información contextual asociada a partir de esta información alimentada. Sobre esta base, un modelo generador produce reglas YARA fundamentadas en la evidencia recopilada, y un mecanismo de self-reflection refina automáticamente la regla generada para mejorar su precisión y reducir falsos positivos. Como resultado, se reduce considerablemente el tiempo de generación y refinamiento de las reglas, lo que aumenta de forma notable la productividad del analista y le permite centrarse en tareas de mayor valor estratégico. Esto permite democratizar la capacidad de crear detecciones complejas al permitir que cualquier analista, incluso sin mucha experiencia en el área, pueda generar reglas YARA avanzadas. Con este sistema RAG ese proceso se vuelve mucho más accesible y rápido. Sin embargo, las reglas generadas aun necesitan ser verificadas para evitar falsos positivos o errores y también es importante que los analistas entiendan cómo funciona la detección de malware para evitar la dependencia de la herramienta.

4. PRINCIPALES APORTES

4.1 Aporte metodológico

Uno de los principales aportes del proyecto consiste en la aplicación de arquitecturas RAG en la generación de reglas YARA. En vez de usar los métodos comunes que se basan en reglas construidas manualmente o generación de reglas directamente mediante los modelos de lenguaje, el sistema propuesto introduce una capa de recuperación de conocimiento que permite:

- Fundamentar la generación en evidencia real
- Mejorar la trazabilidad de las reglas generadas
- Reducir la dependencia del conocimiento (basado en la cantidad de parámetros de entrenamiento) implícito del modelo de lenguaje

Este enfoque representa un avance significativo hacia la generación asistida y controlada de reglas YARA, acercándose a prácticas más rigurosas en inteligencia de amenazas.

4.2 Aporte técnico

El desarrollo de un script para el análisis estático en Python constituye otro aporte relevante, ya que permite extraer automáticamente características clave de muestras de malware como:

- Strings relevantes
- Librerías (.dll, .so)
- Ejecutables asociados
- APIs utilizadas
- Indicadores de compromiso (IoCs)
- Entropía de secciones

Al estar automatizado, este procedimiento disminuye el trabajo manual de un analista de ciberseguridad, y además organiza la información de un modo que el sistema RAG pueda usarla. Además, este script podría ser modificado para mejorarlo u adicionar nuevas características de malware las cuales considere la persona especialista.

4.3 Aporte comparativo

Otro aporte importante es la comparación entre diferentes modelos de lenguaje teniendo en cuenta modelos generalistas y enfoques con y sin la implementación del RAG.

Este análisis permite identificar comportamientos característicos, tales como:

- Modelos con alto recall pero mayor tasa de falsos positivos
- Modelos más precisos pero menos sensibles
- Diferencias en estabilidad y consistencia

Estos hallazgos contribuyen al entendimiento del uso de LLM en tareas de ciberseguridad.

4.4 Aporte en escalabilidad

Se plantea como futuro trabajo y recomendación el uso de modelos de lenguaje a gran escala (entrenados con la mayor cantidad de parámetros) como DeepSeek 671B, analizando su impacto en:

- Calidad de generación de las reglas YARA
- Capacidad de razonamiento
- Generalización

4.5 Aporte en soluciones Security Operations Center (SOC)

El presente proyecto fortalece las capacidades operativas de un Security Operations Center (SOC) mediante la automatización del proceso de generación de reglas YARA para detectar malware de tipo ransomware. La integración de análisis estático, arquitecturas RAG y modelos de lenguaje permite reducir significativamente los tiempos asociados al análisis de muestras, extracción de indicadores de compromiso y construcción de mecanismos de detección. Esto puede permitir optimizar actividades de threat hunting, análisis de malware e inteligencia de amenazas, facilitando una respuesta más rápida frente a nuevas variantes de ransomware y disminuyendo el trabajo que recae sobre los analistas.

Adicionalmente, el enfoque propuesto aporta mejor trazabilidad en la generación de reglas YARA, al poder hacer seguimiento a las fuentes de bases de conocimiento especializadas que contiene muestras de malware. El proceso de self-reflection y refinamiento permite mejorar la calidad de las reglas generadas, reduciendo potencialmente falsos positivos y fortaleciendo los procesos de detección dentro del SOC. En general el sistema RAG actúa como una herramienta de apoyo inteligente que incrementa la eficiencia, escalabilidad y capacidad de adaptación de los equipos de ciberseguridad frente a amenazas emergentes.

5. RECOMENDACIONES Y FUTUROS TRABAJOS

A partir de los resultados obtenidos y el análisis desarrollado en esta investigación, se plantean una serie de recomendaciones orientadas a mejorar el sistema propuesto y ampliar su alcance en futuros trabajos.

Dado que el desempeño del modelo depende en gran medida de la información recuperada, resulta fundamental incorporar fuentes más amplias y actualizadas de malware, incluyendo reportes de inteligencia de amenazas, nuevos repositorios y datasets con mayor variedad de familias y variantes. Esto permitiría mejorar la cobertura y la capacidad de generalización de las reglas YARA generadas.

En segundo lugar, se propone integrar técnicas de análisis dinámico de malware como complemento al análisis estático ya implementado. Mientras que el análisis estático permite identificar características estructurales del archivo, el análisis dinámico puede aportar información sobre comportamiento en ejecución, como conexiones de red, creación de procesos y modificación de archivos. La combinación de ambos enfoques enriquecería significativamente el contexto disponible para la generación de reglas (Coscia et al., 2025).

Otra línea de mejora, es optimizar la configuración del proceso de indexación y recuperación en el RAG. Se trata de usar modelos de embeddings que sean más sofisticados o avanzados, modificar los parámetros de cantidad de fragmentos de información se seleccionan (top-k) o el tamaño de esos fragmentos (chunk size), y también usar métodos de re-ranking. Estas mejoras, permitiría recuperar información más relevante y mejorar la calidad del contexto proporcionado al modelo generativo.

Durante esta investigación, nos encontramos con una limitante la cual estaba relacionada con la capacidad computacional que se tenía disponible. Esto condicionó el elegir una versión apta para nuestros requerimientos (DeepSeek-R1:8B). Con este modelo logramos conseguir que el sistema funcionara y obtener resultados relevantes, hay que tener en cuenta que versiones más grandes del modelo, la versión actual de DeepSeek-R1 es la de 671B de datos de entrenamiento, podrían ofrecer mejoras en la calidad de la generación debido a que tiene una mayor capacidad de comprensión del contexto recuperado por el sistema RAG. Implementar un sistema RAG con esta versión del modelo grande de DeepSeek podría ser un gran trabajo debido a que como quedó

demostrado en la evaluación de las reglas YARA generadas por el modelo de DeepSeek-R1:8B estas tienen una gran precisión y una baja tasa de falsos positivos, y con el contexto específico del análisis estático de las muestras de malware podrían generar reglas YARA bastante robustas para la identificación de archivos maliciosos.

Figura 11.

Comparación DeepSeek 8B y DeepSeek 671B. Nota: Imagen generada con ChatGPT (GPT Image, OpenAI, 2026)



Sin embargo, usar modelos de este tamaño (671B) presenta problemas importantes desde el punto de vista de infraestructura. Para que funcione un modelo que tiene más de cien mil millones de parámetros:

- Múltiples tarjetas gráficas (GPU) como las NVIDIA A100 o H100
- Grandes cantidades de memoria RAM y VRAM
- Sistemas de almacenamiento de alta velocidad
- Configuraciones distribuidas tipo clúster

Tabla 7.

Comparación capacidad computacional mínima de DeepSeek-R1 671B vs el hardware del sistema RAG del proyecto.

Componente	Sistema RAG	DeepSeek-R1 671B
CPU	AMD Ryzen 7 7435HS (8 núcleos)	Mínimo 32 núcleos (ejemplo AMD Threadripper 7980X con 64 núcleos)
RAM	16 GB	512 GB DDR5 mínimo
Almacenamiento	500 GB NVMe SSD	1.3–2 TB NVMe SSD
GPU	NVIDIA RTX 4050 (6 GB VRAM)	Multiple A100/H100 (80 GB VRAM each) or 4×RTX 4090

Implementar estos modelos en entornos locales excede las capacidades propias, como las usadas para este proyecto. Como ya se mencionó, para lograr esta implementación se requiere infraestructura especializada.

Finalmente, como trabajo futuro, se propone la creación de un framework automatizado de generación y validación de reglas YARA, que integre todas las etapas desarrolladas en esta investigación: recolección de datos, análisis estático/dinámico, recuperación de información, generación, self-reflection y evaluación. Este framework podría ser utilizado tanto en entornos académicos como en escenarios reales de ciberseguridad.

6. REFERENCIAS

- Alchi, A. N., & Dodiya, K. R. (2023). Demystifying Ransomware: Classification, Mechanism and Anatomy: En K. Kaushik & A. Bhardwaj (Eds.), *Advances in Information Security, Privacy, and Ethics* (pp. 171-192). IGI Global. <https://doi.org/10.4018/978-1-6684-8218-6.ch007>
- Al-Karaki, J., Khan, M. A.-Z., & Omar, M. (2024). *Exploring LLMs for Malware Detection: Review, Framework Design, and Countermeasure Approaches* (arXiv:2409.07587). arXiv. <https://doi.org/10.48550/arXiv.2409.07587>
- Bharti, P., Roy, S. S., & Suresh, A. (2023). Implementation of Yara Rules in Android. *2023 International Conference on Computer Communication and Informatics (ICCCI)*, 1-5. <https://doi.org/10.1109/ICCCI56745.2023.10128288>
- Bilstein, F., & Plohmann, D. (2019). YARA-Signator: Automated Generation of Code-based YARA Rules. *The Journal on Cybercrime and Digital Investigations*, 5(1), 1-13. <https://doi.org/10.18464/CYBIN.V5I1.24>
- Coscia, A., Lorusso, R., Maci, A., & Urbano, G. (2025). APIARY: An API-based automatic rule generator for yara to enhance malware detection. *Computers & Security*, 153, 104397. <https://doi.org/10.1016/j.cose.2025.104397>
- Das, D., Satapathy, S. M., D, A., & Agarwal, A. (2023). Application of Hybrid Approach towards Multi Aspect Classification and Analysis of Malware. *2023 OITS International Conference on Information Technology (OCIT)*, 284-289. <https://doi.org/10.1109/OCIT59427.2023.10430958>
- Jelodar, H., Bai, S., Hamed, P., Mohammadian, H., Razavi-Far, R., & Ghorbani, A. (2025). *Large Language Model (LLM) for Software Security: Code Analysis, Malware Analysis, Reverse Engineering* (arXiv:2504.07137). arXiv. <https://doi.org/10.48550/arXiv.2504.07137>
- Leviathan, Y., Kalman, M., & Matias, Y. (2025). *Prompt Repetition Improves Non-Reasoning LLMs* (Versión 1). arXiv. <https://doi.org/10.48550/ARXIV.2512.14982>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (arXiv:2005.11401). arXiv. <https://doi.org/10.48550/arXiv.2005.11401>
- Li, B., & Zhao, C. (2025). Self-reflection enhances large language models towards substantial academic response. *Npj Artificial Intelligence*, 1(1), 42. <https://doi.org/10.1038/s44387-025-00045-3>

- Li, H., & Li, J. (2024). Automatically Generate Malware Detection Rules By Extracting Risk Information. *Proceedings of the 2024 5th International Conference on Computing, Networks and Internet of Things*, 595-599. <https://doi.org/10.1145/3670105.3670209>
- Li, S., Ming, J., Qiu, P., Chen, Q., Liu, L., Bao, H., Wang, Q., & Jia, C. (2023). PackGenome: Automatically Generating Robust YARA Rules for Accurate Malware Packer Detection. *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 3078-3092. <https://doi.org/10.1145/3576915.3616625>
- Mansour, Z., Ou, W., Ding, S. H. H., Zulkernine, M., & Charland, P. (2025). NeuroYara: Learning to Rank for Yara Rules Generation Through Deep Language Modeling and Discriminative N-Gram Encoding. *IEEE Transactions on Dependable and Secure Computing*, 22(2), 1747-1762. <https://doi.org/10.1109/TDSC.2024.3449641>
- Mohammadinooshan, A., Kargén, U., & Shahmehri, N. (2019). Robust Detection of Obfuscated Strings in Android Apps. *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security*, 25-35. <https://doi.org/10.1145/3338501.3357373>
- Mr. Shreyas Biju Nair, Mr. Laalas Tadavarthy, Mr. Kailas M K, & Mr. Gowrishankar T O. (2024). YARWEB: WEB-BASED GENERIC YARA RULE GENERATOR. *EPRA International Journal of Research & Development (IJRD)*, 295-302. <https://doi.org/10.36713/epra15953>
- Naik, N., Jenkins, P., Cooke, R., Gillett, J., & Jin, Y. (2020). Evaluating Automatically Generated YARA Rules and Enhancing Their Effectiveness. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 1146-1153. <https://doi.org/10.1109/SSCI47803.2020.9308179>
- Naik, N., Jenkins, P., Savage, N., Yang, L., Boongoen, T., lam-On, N., Naik, K., & Song, J. (2021). Embedded YARA rules: Strengthening YARA rules utilising fuzzy hashing and fuzzy rules for malware analysis. *Complex & Intelligent Systems*, 7(2), 687-702. <https://doi.org/10.1007/s40747-020-00233-5>
- Naik, N., Jenkins, P., Savage, N., Yang, L., Naik, K., & Song, J. (2019). Augmented YARA Rules Fused With Fuzzy Hashing in Ransomware Triaging. *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, 625-632. <https://doi.org/10.1109/SSCI44817.2019.9002773>
- Naik, N., Jenkins, P., Savage, N., Yang, L., Naik, K., & Song, J. (2020). Embedding Fuzzy Rules with YARA Rules for Performance Optimisation of Malware Analysis. *2020 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE)*, 1-7. <https://doi.org/10.1109/FUZZ48607.2020.9177856>
- Ocănoaia, A.-I., Mihăilescu, M.-E., Năstase, V.-I., Jumărea, Ș.-D., Carabaș, M., Dagilis, L. V., & Sanda, D. (2025). From Incident to IOC - An Automated Malware Investigation Pipeline. *2025 24th*

- RoEduNet Conference: Networking in Education and Research (RoEduNet)*, 1-5. <https://doi.org/10.1109/RoEduNet68395.2025.11208341>
- Pal Singh Shobhit, J. (2024). Automating Threat Intelligence Analysis with Retrieval Augmented Generation (RAG) for Enhanced Cybersecurity Posture. *International Journal of Science and Research (IJSR)*, 13(5), 251-255. <https://doi.org/10.21275/SR24502103758>
- Patel, P. S., Kunwar, R. S., & Thakar, A. (2023). Malware Detection Using Yara Rules in SIEM: En S. L. Shiva Darshan, M. V. Manoj Kumar, B. S. Prashanth, & Y. Vishnu Srinivasa Murthy (Eds.), *Advances in Information Security, Privacy, and Ethics* (pp. 313-330). IGI Global. <https://doi.org/10.4018/978-1-6684-8666-5.ch014>
- Patil, V., M, N. K., M, P. S., & Singh, A. (2025). Effectively Writing YARA Rules to Detect Malware. *International Journal for Research in Applied Science and Engineering Technology*, 13(1), 1265-1273. <https://doi.org/10.22214/ijraset.2025.66535>
- Raff, E., Sylvester, J., & Nicholas, C. (2017). *Learning the PE Header, Malware Detection with Minimal Domain Knowledge*. <https://doi.org/10.48550/ARXIV.1709.01471>
- Raff, E., Zak, R., Lopez Munoz, G., Fleming, W., Anderson, H. S., Filar, B., Nicholas, C., & Holt, J. (2020). Automatic Yara Rule Generation Using Biclustering. *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 71-82. <https://doi.org/10.1145/3411508.3421372>
- Renze, M., & Guven, E. (2024). The Benefits of a Concise Chain of Thought on Problem-Solving in Large Language Models. *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, 476-483. <https://doi.org/10.1109/FLLM63129.2024.10852493>
- Si, Q., Xu, H., Tong, Y., Zhou, Y., Liang, J., Cui, L., & Hao, Z. (2022). Malware Detection Using Automated Generation of Yara Rules on Dynamic Features. En C. Su, K. Sakurai, & F. Liu (Eds.), *Science of Cyber Security* (Vol. 13580, pp. 315-330). Springer International Publishing. https://doi.org/10.1007/978-3-031-17551-0_21
- Siddabathula, K. S., Panneerselvam, R. K., Vasana, V., Vejendla, J., Rafi, M., & Gummadi, S. B. (2023). YaraCapper – YARA Rule-Based Automated System to Detect and Alert Network Attacks. En A. Verma, P. Verma, K. K. Pattanaik, & L. Garg, *Research Advances in Network Technologies* (1.^a ed., pp. 25-47). CRC Press. <https://doi.org/10.1201/9781003320333-3>
- Wang, X., Liu, Y., Wang, X., & Cheng, C. (2025). *LLMDYara: LLMs-Driven Automated YARA Rules Generation with Explainable File Features and DNAHash*.
- Xu, L., & Qiao, M. (2022). Yara rule enhancement using Bert-based strings language model. *2022 5th International Conference on Advanced Electronic Materials, Computers and Software Engineering (AEMCSE)*, 221-224. <https://doi.org/10.1109/AEMCSE55572.2022.00052>

Zhang, X., Chen, H., He, Y., Niu, W., & Li, Q. (2025). *Automatically Generating Rules of Malicious Software Packages via Large Language Model*. <https://doi.org/10.48550/ARXIV.2504.17198>